SĂPTĂMÂNA 7

Sisteme dinamice continue și integrare numerică

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Datele de intrare se aleg din date: Q-Q, KS, χ^2, AIC; forma distribuției poate schimba rezultatul cu o treime (S6)
- ✓ O rulare = o observație: **replicări**, încălzire, intervale de încredere, numere aleatoare comune
- ✓ Verificare (programul e corect) vs **validare** (modelul e potrivit scopului, pe mărimea de interes)
- ✓ Întrebarea de final: ce facem când starea se schimbă **tot timpul** și nu mai există „următorul eveniment”?

La finalul acestei săptămâni veți putea…

1. să descrieți un sistem prin **variabile de stare**, intrări, ieșiri și parametri și să scrieți o **ecuație de bilanț**
2. să construiți modele de **creștere** (exponențială, logistică), de răcire și de tip **stocuri și fluxuri**
3. să rezolvați numeric o ecuație diferențială cu metodele **Euler** și **Runge–Kutta 4** și să alegeți pasul de timp
4. să explicați **eroarea** locală și globală, **stabilitatea** și sistemele rigide; să folosiți solve_ivp
5. să simulați modele **Lotka–Volterra** și **SIR/SEIR** și să interpretați R_0
6. să **calibrați** parametrii unui model pe date reale și să discutați identificabilitatea și incertitudinea

Evenimente discrete vs dinamică continuă

Evenimente discrete (S5)

- starea sare la momente izolate: o sosire, o plecare
- între evenimente starea este **constantă**
- simulatorul sare la următorul eveniment
- entități individuale: clienți, pachete, mașini

vs

Sisteme continue (S7)

- starea se schimbă **în fiecare clipă**: temperatura, populația, viteza
- modelul dă **rata de schimbare**: dx/dt = f(x, t)
- simulatorul avansează cu un **pas de timp** h
- mărimi agregate: concentrații, populații, stocuri

Multe sisteme reale sunt **hibride**: un termostat (pornit/oprit = evenimente) controlează o temperatură (continuă); o mașină autonomă are dinamică continuă și decizii discrete. Bila din S5 era un caz particular: între ciocniri, mișcarea avea soluție exactă.

Unde apar sisteme continue în informatică

1 Hardware

Temperatura procesorului, descărcarea bateriei, circuite analogice: ecuații RC.

2 Rețele

Controlul congestiei TCP descris ca **model fluid**: fereastra crește liniar și scade la pierderi.

3 Sisteme distribuite

Protocoalele „gossip” răspândesc informația exact ca o epidemie (modelul SI/SIR).

4 Jocuri și grafică

Motoarele fizice integrează legile lui Newton de 60 de ori pe secundă (S8).

5 Produse și utilizatori

Adoptarea unei aplicații, abandonul, efectul de rețea: modele de tip Bass.

6 Învățare automată

Coborârea pe gradient este metoda Euler pe un flux de gradient; neural ODE (2018) învață chiar f.

Ora 1

Modele matematice ale dinamicii

1. Stare, intrări, ieșiri, parametri (Holban cap. 6)
2. Recurențe și ecuații diferențiale; ecuația de bilanț
3. Creștere exponențială și logistică; răcirea lui Newton
4. Dinamica sistemelor: stocuri, fluxuri, bucle de reacție

Modelul matematic al unui sistem dinamic

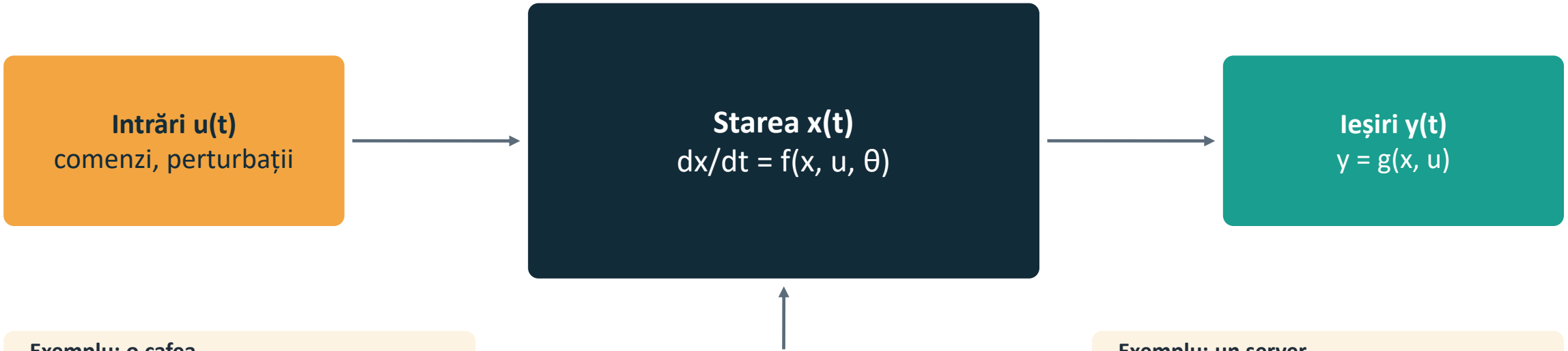
Model matematic (Holban, cap. 6)

Un sistem de relații sau **ecuații diferențiale** care descrie, într-o măsură rezonabilă, comportamentul unui sistem real. **Simularea** înseamnă rezolvarea acestor ecuații în condiții precizate și compararea permanentă a rezultatelor cu sistemul real.

Holban cap. 6; Cernazean L3

- Variabilele de **intrare** u(t) sunt de obicei manipulabile; **ieșirile** y(t) sunt ceea ce observăm
- Starea x(t) conține informația **minimă** necesară pentru a prezice evoluția viitoare
- Ieșirile depind atât de intrări, cât și de **starea inițială** $x(t_o)$
- **Parametrii** fac modelul particular: îl pun în acord cu un sistem fizic concret

Intrări, stare, ieșiri, parametri



Exemplu: o cafea
intrare: temperatura camerei
stare: temperatura cafelei
parametru: k (cât de repede se răcește)

Exemplu: un server
intrare: cererile pe secundă
stare: temperatura procesorului
ieșire: frecvența (throttling)

Două descrieri ale aceleiași dinamici (Cernazean L3)

Recurență (timp discret)

- $s(t + \Delta t) = F(s(t))$
- starea se calculează **din pas în pas**
- naturală pentru date anuale, generații, iterații
- se implementează direct pe calculator

Ecuație diferențială (timp continuu)

- $ds/dt = f(s(t))$
- dă **rata de schimbare** în fiecare clipă
- naturală pentru fizică, chimie, epidemii
- se rezolvă exact rar; de obicei **numeric**

Legătura: $(s(t + \Delta t) - s(t))/\Delta t \to ds/dt$ când $\Delta t \to 0$. Invers, cea mai simplă rezolvare numerică a unei ecuații diferențiale este chiar o recurență: $s(t + \Delta t) \approx s(t) + \Delta t \cdot f(s(t))$ – **metoda Euler** din ora 2.

Ecuația de bilanț: rețeta oricărui model

$$dS/dt = \text{rata de creare} - \text{rata de distrugere}$$

populație	dP/dt = nașteri − decese = b·P − m·P = r·P (b, m: rate de natalitate și mortalitate)
cont bancar	dB/dt = dobândă + depuneri − retrageri
coadă (fluid)	dQ/dt = rata sosirilor − rata servirilor (aproximare continuă a unei cozi foarte mari)

Pas cu pas: (1) alegem mărimile care se acumulează (stările); (2) pentru fiecare, listăm ce o crește și ce o scade; (3) scriem fiecare rată în funcție de stări și parametri. Restul este matematică și calcul.

Reducerea la un sistem de ordinul întâi

$$x'' = -\omega^2 \cdot x \quad \Longrightarrow \quad x' = v, \quad v' = -\omega^2 \cdot x$$

ideea	introducem derivatele ca **variabile de stare noi**: aici viteza v = x′
general	o ecuație de ordinul n devine un sistem de n ecuații de ordinul 1 (Holban, cap. 6): $x_1 = y$, $x_2 = y'$, …, $x_n = y^{(n-1)}$
dependență de t	dacă f depinde explicit de timp, adăugăm τ cu τ′ = 1 (Cernazean L3)

Consecința practică: un singur integrator, scris pentru **s′ = f(s)** cu s vector, rezolvă orice sistem – un arc, un pendul, o orbită, o epidemie. Exact așa lucrează solve_ivp din SciPy.

Forma generală și ce garantează matematica

- Problema Cauchy: **$s'(t) = f(s(t), t, \theta)$**, cu starea inițială $s(t_0) = s_0$ cunoscută
- Dacă f este „netedă” (lipschitziană), soluția **există și este unică** (teorema Picard–Lindelöf): același start, același viitor
- Determinism nu înseamnă predictibilitate: unele sisteme amplifică erorile mici exponențial (**haos**, vom vedea imediat)
- Soluții exacte există doar pentru modele simple: exponențială, logistică, răcire, oscilatorul liniar
- Lotka–Volterra, SIR, vremea, orbitele a trei corpuri: **nu** au formulă – le rezolvăm numeric (ora 2)

S **Spațiul stărilor**

Fiecare stare posibilă este un punct; soluția este o **traiectorie** prin acest spațiu. Graficele „pradă vs prădător” din ora 2 sunt astfel de traiectorii.

Primele modele

Creștere: exponențială și logistică

1. Creșterea exponențială și timpul de dublare
2. Discret vs continuu: dobânda compusă
3. Capacitatea mediului: modelul logistic
4. Echilibre, stabilitate și o surpriză: haosul

Creșterea exponențială

$$dP/dt = r \cdot P \quad \Longrightarrow \quad P(t) = P_0 \cdot exp(r \cdot t)$$

$r = b - m$	rata netă de creștere: nașteri minus decese pe individ (Cernazean L3)
timp de dublare	$T_2 = \ln 2 / r \approx 0{,}693/r$: la r = 10% pe an, **6,93 ani** („regula lui 70”: 70/10 = 7)
$r < 0$	declin exponențial: dezintegrare radioactivă, eliminarea unui medicament, uitarea

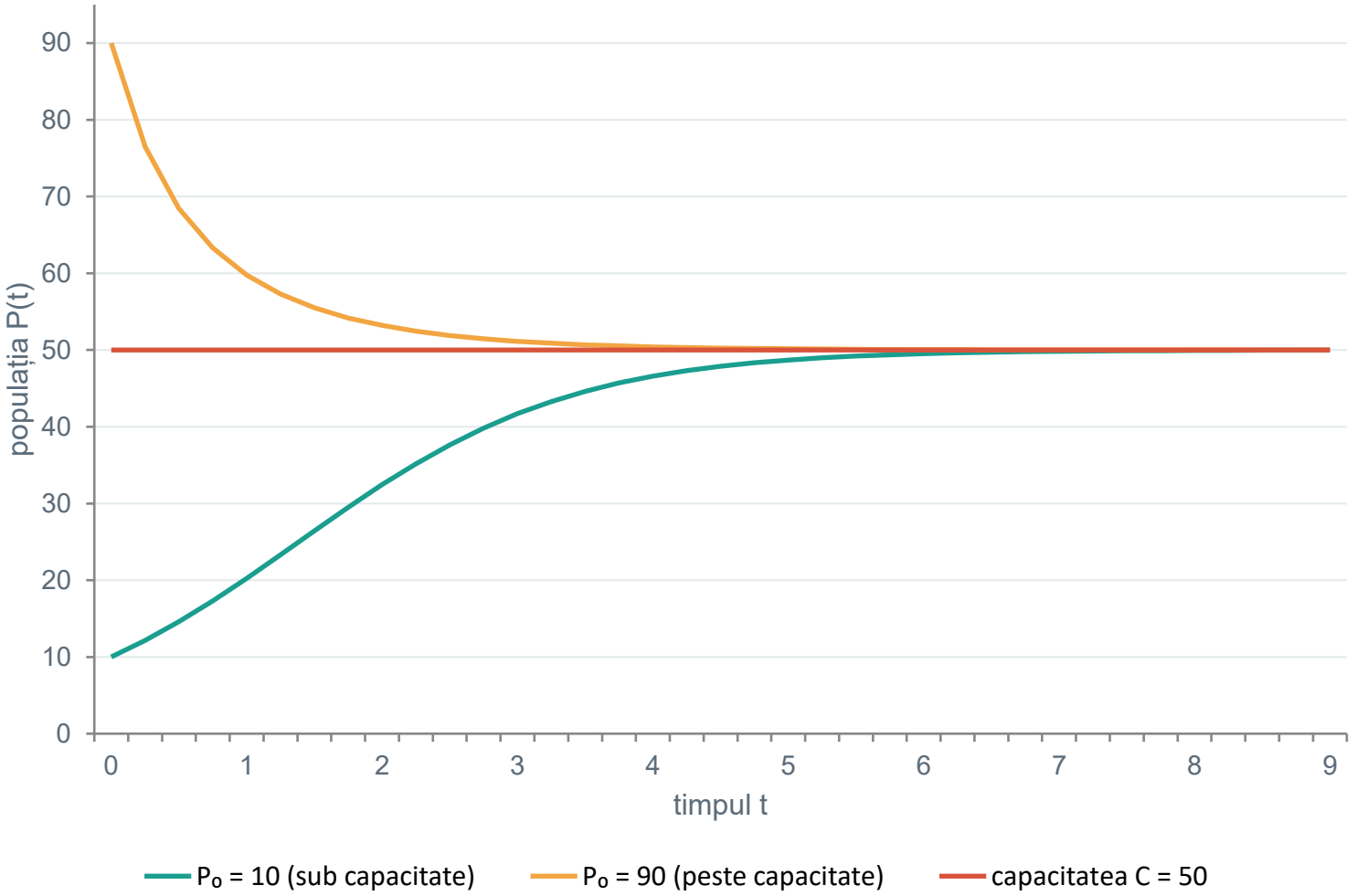
Exemple din informatică: numărul de tranzistori pe cip (legea lui Moore, dublare la ~2 ani), traficul unui serviciu viral, costul de antrenare al modelelor AI. Nicio creștere exponențială nu durează la nesfârșit.

Discret sau continuu? Aceeași rată, rezultate diferite

Modelul	100 după 10 ani, r = 10%	Interpretare
Recurență anuală: P(n+1) = 1,1·P(n)	259,4	creșterea se adaugă o dată pe an
Ecuație diferențială: P′ = 0,1·P	271,8	creșterea se adaugă continuu
Diferența	12,4	≈ 5% după 10 ani
Legătura	1 + r ≈ e^r	pentru r mic cele două coincid

- Discretizarea **schimbă** modelul: 10% pe an „compus anual” nu este același lucru cu 10% pe an „compus continuu”
- Aceeași observație stă la baza erorii metodei Euler: înlocuim creșterea continuă cu pași discreți

Modelul logistic: C = 50, v = 1 (Cernazean L3)



- De jos: o curbă în formă de **S**
- De sus: scădere spre C
- Ambele ajung la **C = 50**
- Jumătatea capacității: la $t = \ln 4 \approx 1{,}39$

Ecuația logistică (Verhulst, 1838)

$$dP/dt = v \cdot P \cdot (1 - P/C)$$

ideea	rata de creștere r = v·(1 − P/C) scade la 0 când P se apropie de capacitatea **C**
soluția	P(t) = C / (1 + ((C − P_0)/P_0)·e^(−v·t))
echilibre	P = 0 (**instabil**: orice populație mică crește) și P = C (**stabil**: atrage toate traiectoriile)

Aceeași curbă în S apare la adoptarea tehnologiilor, la creșterea utilizatorilor unei rețele sociale, la vânzările unui produs nou și la învățarea unei abilități. Laboratorul de creștere a populației folosește exact acest model.

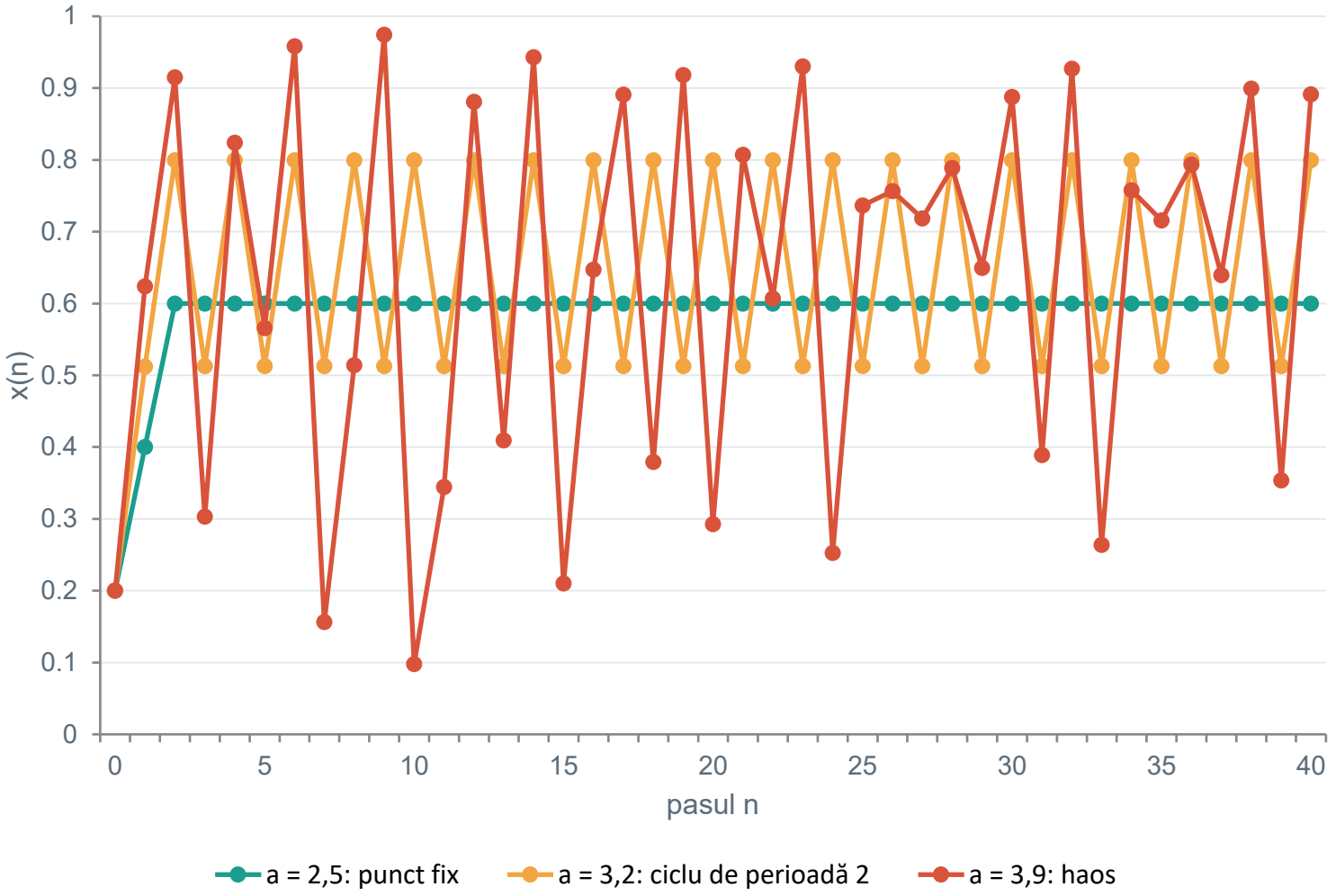
Echilibre și stabilitate

- Un **echilibru** (punct fix) x* este o stare în care f(x*) = 0: sistemul rămâne acolo pentru totdeauna
- **Stabil**: o mică perturbare se stinge, sistemul revine; **instabil**: perturbarea crește
- Testul pentru o ecuație scalară: semnul lui **f’(x*)** – negativ = stabil, pozitiv = instabil
- Logistica: f’(0) = v > 0 (instabil), f’(C) = −v < 0 (stabil)
- Pentru sisteme: valorile proprii ale matricei Jacobi (părțile reale negative = stabil); centrele (Lotka–Volterra) sunt cazuri limită

B **Intuiția**

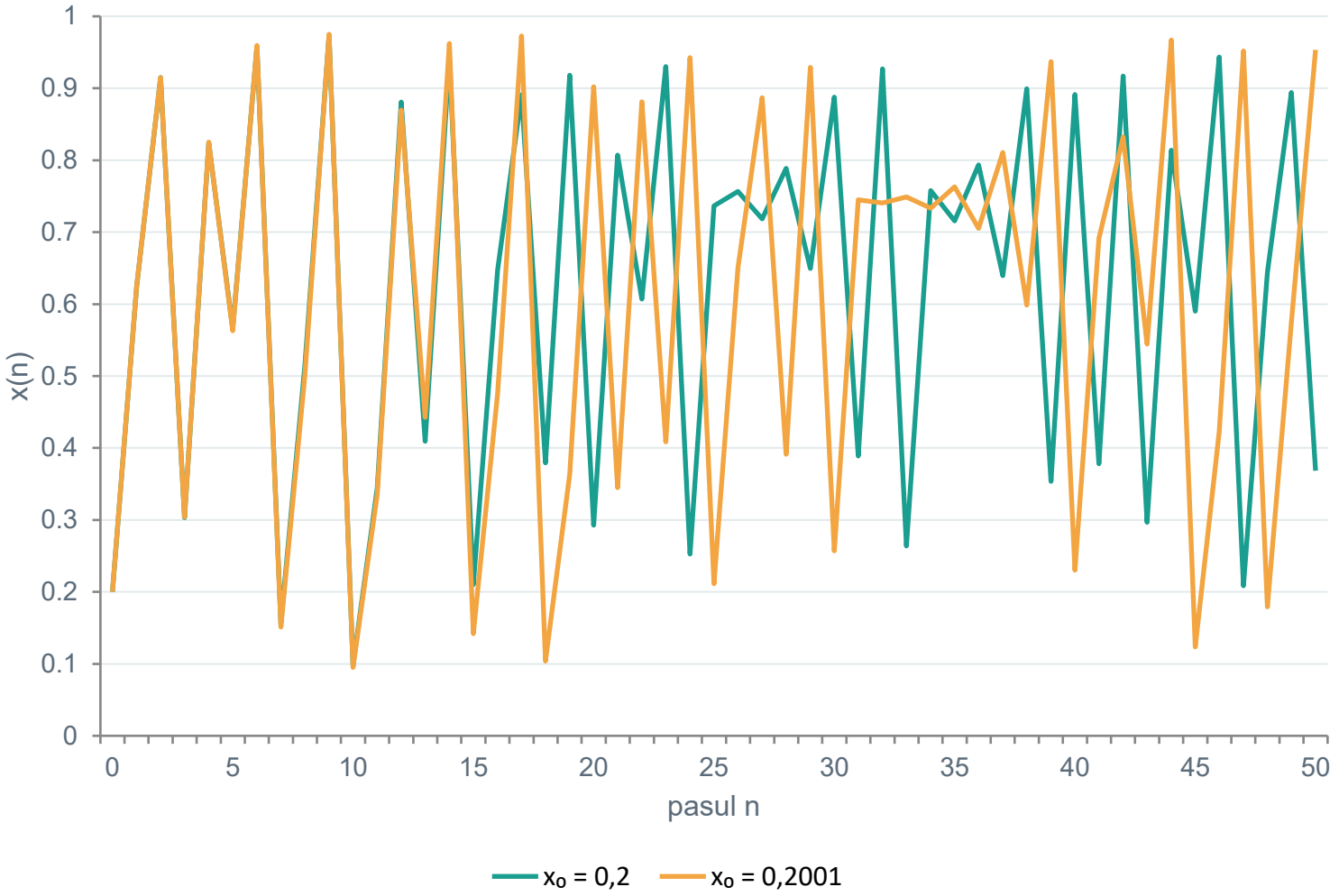
O bilă într-o vale (stabil) și o bilă pe vârful unui deal (instabil): ambele sunt echilibre, dar doar una supraviețuiește unei mici adieri.

Surpriza: harta logistică discretă x(n+1) = a·x(n)·(1 – x(n))



- Aceeași formă ca logistica, dar **discretă**
- a = 2,5: converge la 0,6
- a = 3,2: oscilează între 0,51 și 0,80
- a = 3,9: **niciodată** nu se repetă (May, 1976)

Haos: sensibilitate la condițiile inițiale (a = 3,9)



- Diferență inițială: **0,0001**
- După 16 pași: diferență de peste 0,1
- Apoi traiectoriile nu mai au nicio legătură
- Lorenz (1963): de aici limita de ~2 săptămâni a prognozei meteo

Verificare

? Pentru modelul $dx/dt = x \cdot (1 - x/50)$, ce se întâmplă dacă pornim din $\mathbf{x_0 = 0{,}5}$?

A x rămâne 0,5 pentru totdeauna

B x scade spre 0

C x crește și se apropie de 50

D x crește fără limită

Un model cu soluție exactă

Răcirea lui Newton

1. Ecuația și soluția
2. Cât timp până bem cafeaua?
3. Estimarea parametrului din două măsurători
4. Aplicație: temperatura unui procesor

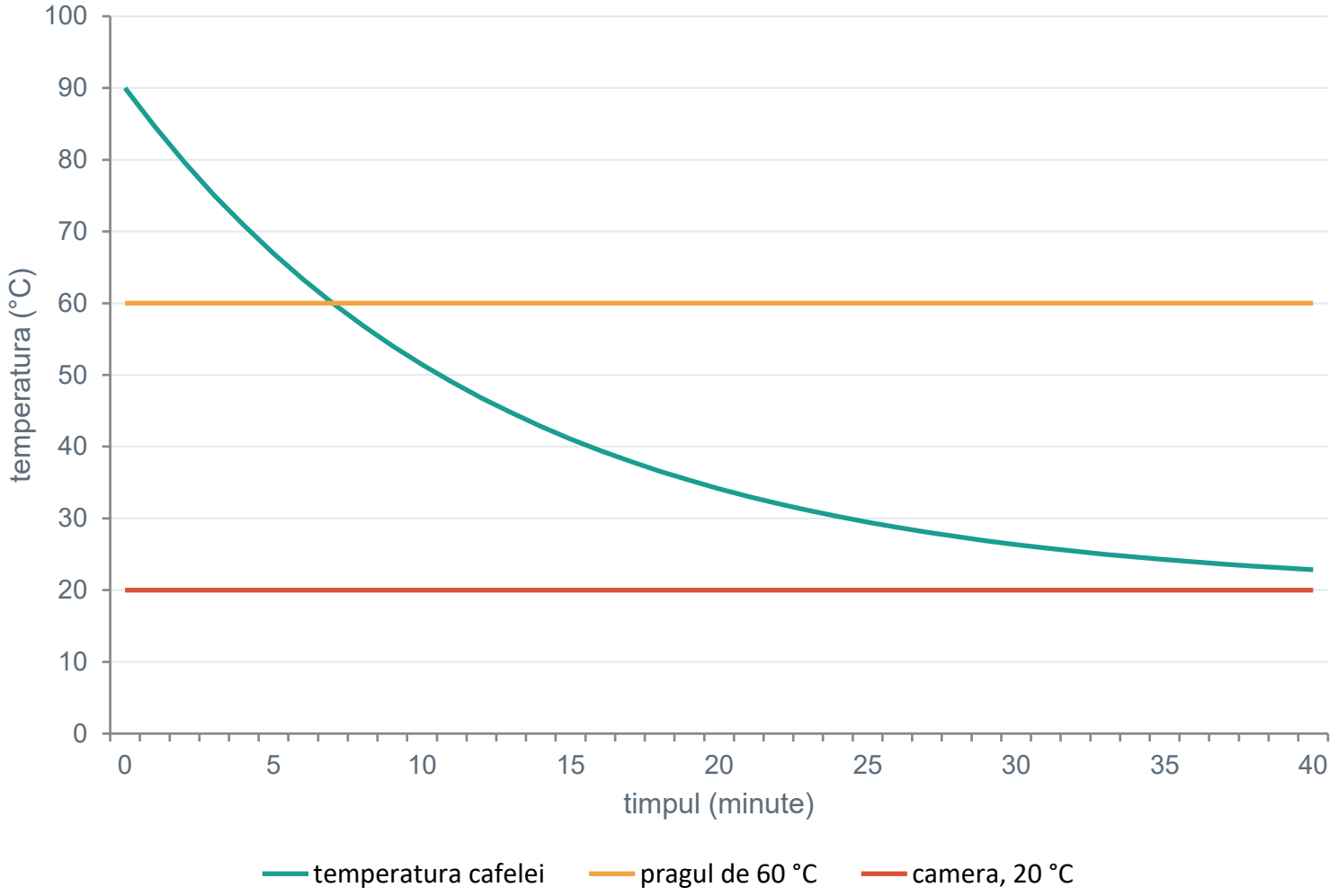
Legea răcirii lui Newton

$$dT/dt = -k\cdot(T - T_a) \quad \Longrightarrow \quad T(t) = T_a + (T_0 - T_a)\cdot exp(-k\cdot t)$$

ideea	un corp pierde căldură cu atât mai repede cu cât este mai cald decât mediul (temperatura $\mathbf{T_a}$)
exemplul	cafea la $\mathbf{T_0 = 90\ °C}$, cameră la $\mathbf{T_a = 20\ °C}$, $\mathbf{k = 0{,}08}$ pe minut
până la 60 °C	$t = \ln((90 - 20)/(60 - 20)) / 0{,}08 = \ln 1{,}75 / 0{,}08 \approx$ **7,0 minute**

Echilibrul $T = T_a$ este stabil ($f' = -k < 0$). Același model descrie încărcarea unui condensator, un medicament eliminat din sânge sau o temperatură a unui procesor după o sarcină de lucru.

Cafeaua se răcește: $T_0 = 90\ °C$, $T_a = 20\ °C$, $k = 0{,}08/\text{min}$



- 60 °C după **7,0 min**
- După 10 min: **51,5 °C**
- Diferența față de cameră scade cu ~7,7% pe minut
- Nu ajunge niciodată exact la 20 °C

Estimarea lui k din date și o aplicație din informatică

- Din două măsurători: $k = \ln((T_1 - T_a)/(T_2 - T_a)) / (t_2 - t_1)$ – de exemplu 90 °C la t = 0 și 51,5 °C la t = 10 dau k = 0,08
- Din multe măsurători: $\ln(T - T_a)$ este o **dreaptă** în timp; panta ei este $-k$ (regresie liniară)
- Aceasta este prima noastră **calibrare**: parametrul se alege astfel încât modelul să reproducă datele (ora 3)
- **Procesoare**: modelul termic RC al unui cip (rezistență și capacitate termică) este exact această ecuație, cu un termen de încălzire proporțional cu puterea
- Planificatoarele moderne folosesc astfel de modele pentru a decide când reduc frecvența (thermal throttling)

P **Model cu sursă**

$dT/dt = P(t)/C_{th} - (T - T_a)/(R_{th} \cdot C_{th})$: temperatura urcă la o sarcină mare și coboară când sarcina scade – un sistem cu intrare $u(t) = P(t)$.

Forrester, MIT, 1956

Dinamica sistemelor: stocuri și fluxuri

1. Stocuri, fluxuri și bucle de reacție
2. Adoptarea unei aplicații: modelul Bass
3. Întârzierile produc oscilații: stocul unui magazin
4. Unelte: Vensim, Stella, AnyLogic, PySD

Stocuri și fluxuri

Stoc = acumulare; flux = rată

Un **stoc** este o mărime care se acumulează în timp (apa din cadă, utilizatorii unei aplicații, banii din cont). Un **flux** este o rată care umple sau golește stocul (litri pe minut, instalări pe zi). Stocul este integrala fluxurilor: **S(t) = S(0) + ∫ (intrare − ieșire) dt**.

Forrester, Industrial Dynamics, 1961; Sterman, Business Dynamics, 2000

- Test simplu: un stoc se poate măsura la un moment dat (o „fotografie”); un flux are nevoie de un interval („pe zi”)
- Stocurile dau **inerție** sistemului: nu se schimbă instantaneu, chiar dacă fluxurile se schimbă brusc
- Fluxurile depind de stocuri prin **bucle de reacție** (feedback)

Diagrama stoc–flux: utilizatorii unei aplicații



Cum se citește
dreptunghi = stoc
săgeată groasă = flux (adoptări pe zi)
nor = granița modelului
R = buclă de întărire, B = buclă de echilibrare

R: cu cât mai mulți utilizatori, cu atât mai multe recomandări

B: piața se epuizează, potențialii scad

Bucle de reacție și întârzieri

1 Buclă de întărire (R)

Un efect își amplifică propria cauză: dobânda compusă, viralizarea, panica bancară. Produce **creștere exponențială**.

2 Buclă de echilibrare (B)

Sistemul caută o țintă: termostatul, capacitatea mediului, autoscalarea unui cluster. Produce **stabilizare**.

3 Întârzieri

Între acțiune și efect trece timp: comenzi pe drum, incubația unei boli. Combinate cu bucle B produc **oscilații**.

4 Neliniarități

Efectul nu e proporțional cu cauza: saturarea pieței, cozi lângă ρ = 1. Schimbă dominanța buclelor.

5 Dominanța buclelor

La început domină R (creștere), apoi B (saturare): asta explică forma de S a logisticii.

6 Comportament contraintuitiv

Forrester: politicile evidente au adesea efecte opuse, pentru că ignoră buclele și întârzierile.

O scurtă istorie a dinamicii sistemelor

1956 — **J. Forrester** (inventatorul memoriei cu ferite) începe la MIT studiul organizațiilor ca sisteme cu reacție

1961 — **Industrial Dynamics**: lanțuri de producție și distribuție, oscilațiile stocurilor

1969 — **Urban Dynamics**: dezvoltarea și declinul orașelor

1972 — **Limits to Growth** (Meadows et al.): modelul World3, populație–resurse–poluare

1985–90 — **Stella**, **Vensim**: modelare grafică pe PC; „Beer Game" predat în școlile de business

azi — politici de sănătate, climă (En-ROADS), strategie de produs; **PySD** rulează modele Vensim în Python

Un exemplu de model folosit în dezbateri publice: World3 a fost criticat și apărat timp de 50 de ani – validarea modelelor de politici este dificilă.

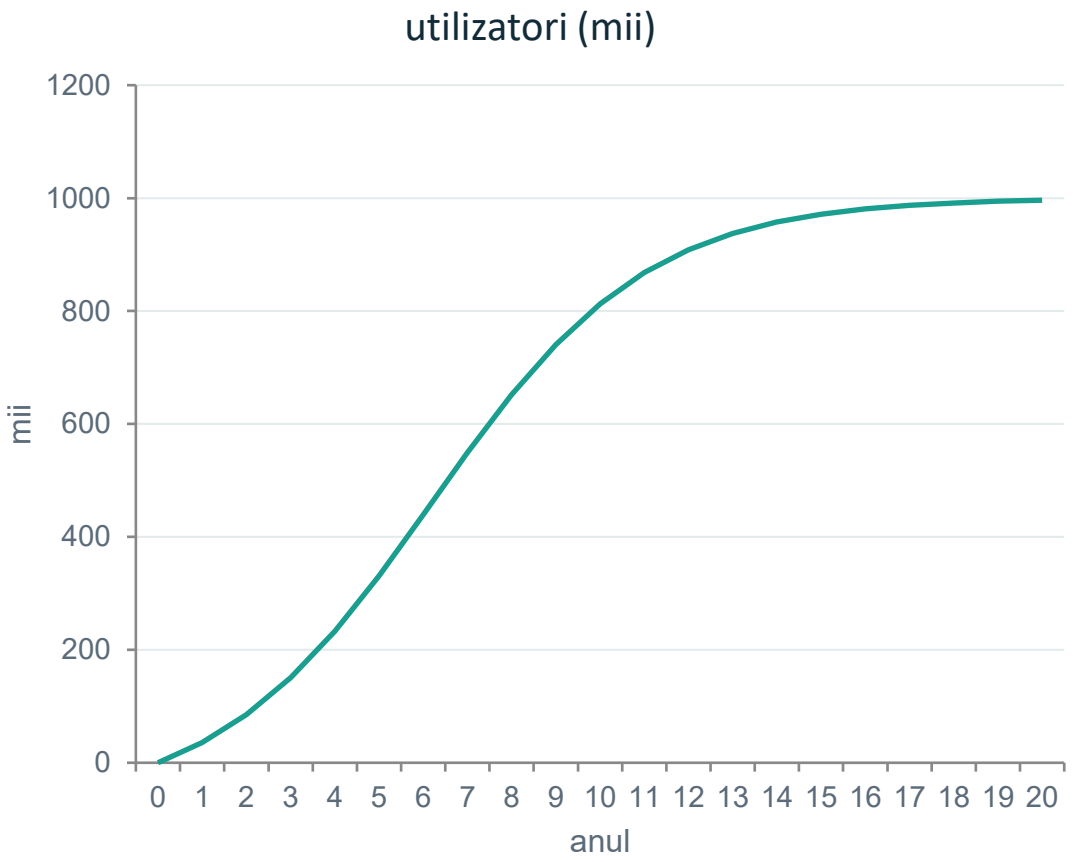
Modelul Bass: cum se răspândește o aplicație nouă

$$dN/dt = (p + q \cdot N/M) \cdot (M - N)$$

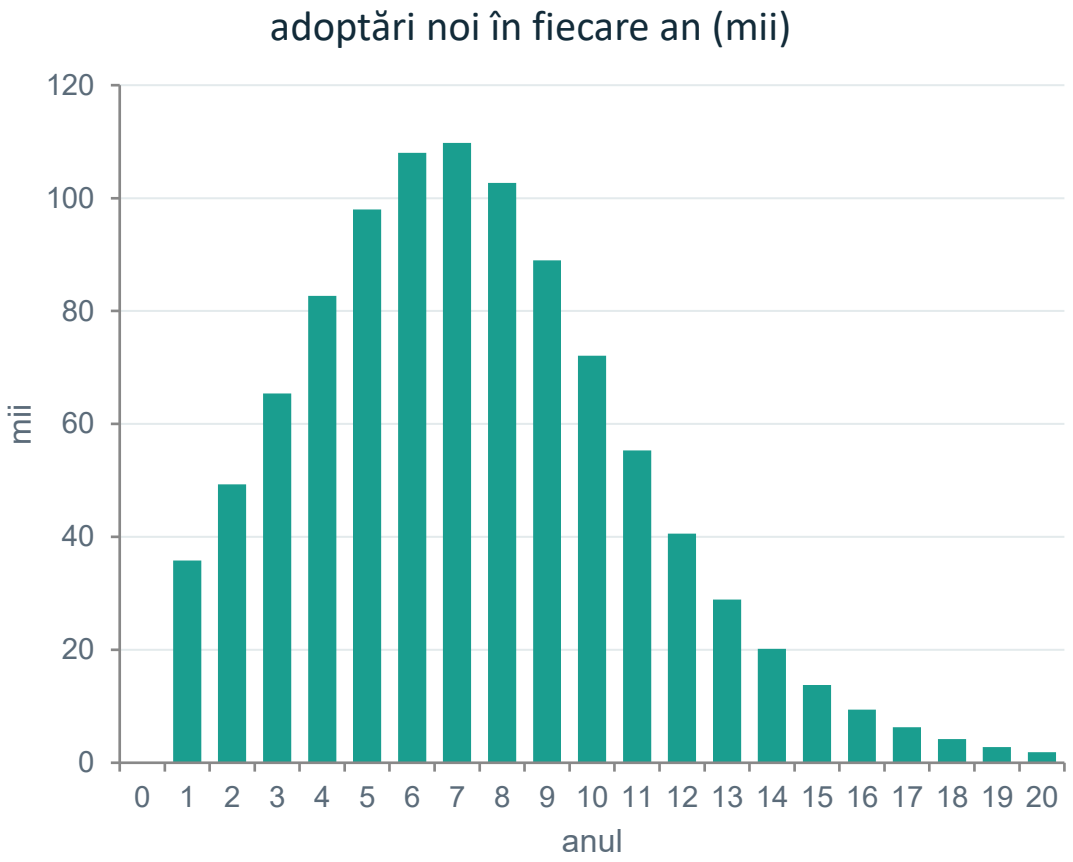
M	piața potențială: **1 milion** de utilizatori
$p = 0{,}03$	**inovare**: adoptări din reclame, independente de ceilalți (rata pe an)
$q = 0{,}38$	**imitare**: adoptări din recomandări, proporționale cu cei care folosesc deja aplicația

Frank Bass (1969) a modelat vânzările bunurilor de folosință îndelungată (televizoare, frigidere); modelul este folosit și azi pentru prognoza adoptării produselor tehnologice. Valorile p ≈ 0,03 și q ≈ 0,38 sunt tipice în studiile empirice.

Modelul Bass: utilizatori cumulați și adoptări noi pe an



curba în **S**: 813 mii după 10 ani



vârful adoptărilor la **t ≈ 6,2 ani**

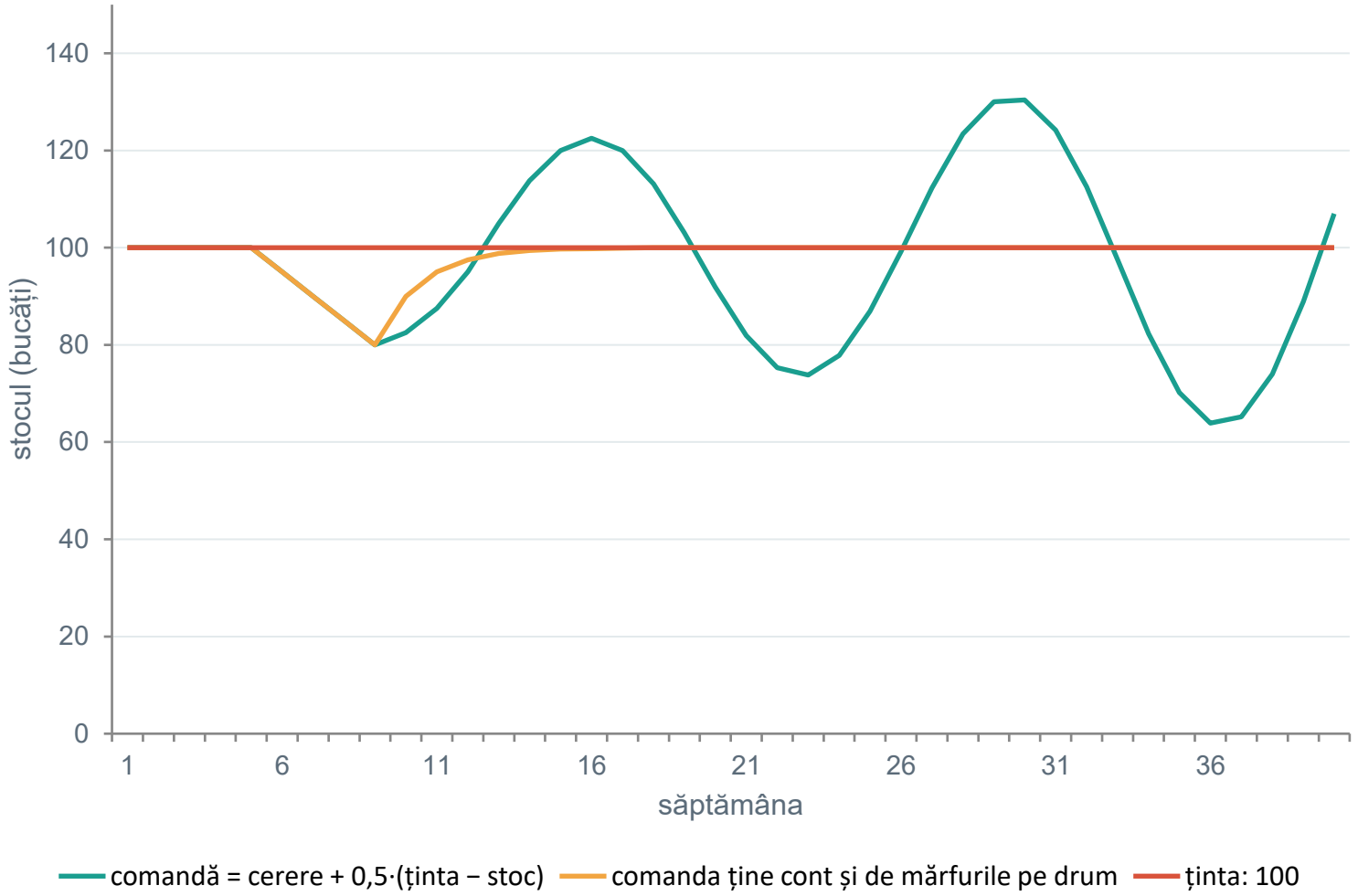
Ce aflăm din modelul Bass

- Vârful adoptărilor: $t^* = \ln(q/p)/(p + q) \approx$ **6,2 ani**, când $N = M\cdot(q - p)/(2q) \approx$ **461 mii**
- Raportul q/p mare = produs „social” (rețele, mesagerie): start lent, apoi explozie; q/p mic = produs adoptat individual
- Planificare: capacitatea serverelor, echipa de suport și bugetul de marketing urmăresc **curba adoptărilor**, nu doar totalul
- Calibrare: p, q și M se estimează din primele luni de date – dar, ca la epidemii, datele timpurii determină slab vârful (ora 3)
- Variante: abandonul utilizatorilor (flux de ieșire), generații succesive de produse, prețul ca intrare

E **Legătura cu SIR**

Termenul $q\cdot N\cdot(M - N)/M$ este exact „contagiunea” din modelul epidemic SIR: ideile, produsele și virusurile se răspândesc prin contact.

Întârzierile produc oscilații: stocul unui magazin



- Cererea crește din săptămâna 6: 20 → 25
- Comenzile sosesc după **4 săptămâni**
- Regula simplă: oscilații **crescătoare** (64 – 130)
- Regula care vede comenzile pe drum: revine lin la 100

Lecțiile stocului: efectul de bici (bullwhip)

- Cauza oscilațiilor nu este cererea (care crește o singură dată), ci **structura**: buclă de echilibrare + întârziere
- Managerul vede stocul mic, comandă mult; comenzile sosesc când problema a trecut deja → stoc prea mare
- În lanțuri cu mai multe niveluri (magazin, distribuitor, fabrică), oscilațiile **cresc** spre amonte: efectul de bici
- „Beer Game” (MIT, anii 1960): jucătorii reproduc aceste oscilații chiar dacă știu că urmează să apară
- Soluția: informație despre **comenzile pe drum** și reacții mai blânde – nu o mai bună prognoză a cererii

A **În informatică**

Autoscalarea unui cluster cu pornire lentă a mașinilor (minute) poate oscila la fel: adaugă prea multe mașini, apoi le oprește. Soluția este aceeași: ține cont de mașinile care pornesc deja.

Dinamica sistemelor, evenimente discrete sau agenți?

	Dinamica sistemelor	Evenimente discrete	Agenți (S9)
Unitatea	stocuri agregate	entități în cozi	indivizi autonomi
Timpul	continuu (EDO)	evenimente	pași sau evenimente
Aleatorul	de obicei determinist	esențial	adesea esențial
Nivelul	strategic, politici	operațional	comportamental, spațial
Exemplu	adoptarea unei aplicații	call center	răspândirea pe o rețea socială

- Aceeași problemă poate fi modelată în toate trei feluri; alegerea depinde de **întrebare** și de nivelul de detaliu
- AnyLogic permite combinarea lor într-un singur model (multimetodă)

Un model stoc–flux în Python: zece linii

```
M, p, q = 1_000_000, 0.03, 0.38
dt, T = 0.01, 20                  # pas de 0,01 ani
N, t, istoric = 0.0, 0.0, []

while t < T:
    adoptari = (p + q * N / M) * (M - N)    # flux
    N += adoptari * dt                      # stoc
    t += dt
    istoric.append((t, N))

print(round(N))   # ≈ 996 000 după 20 de ani
```

- Fluxul se calculează **din stocuri**
- Stocul se actualizează cu flux × dt
- Exact așa lucrează Vensim și Stella
- Este metoda **Euler** – ora 2 arată când nu ajunge

Verificare

? Într-un model al unei platforme de streaming, care dintre mărimi este un **stoc**?

A numărul de abonamente noi pe zi

B numărul de abonați activi

C numărul de anulări pe săptămână

D traficul în gigabiți pe secundă

Unelte pentru simularea continuă

- **Vensim**, **Stella**, **Insight Maker**: diagrame stoc–flux, pentru dinamica sistemelor
- **AnyLogic**: dinamica sistemelor + evenimente discrete + agenți în același model
- **Modelica** (OpenModelica, Dymola) și **Simulink**: sisteme fizice – circuite, motoare, mașini, clădiri
- **Python**: SciPy (solve_ivp), PySD (modele Vensim), JAX/Diffrax (ecuații diferențiale diferențiabile, pentru învățare automată)
- Toate au în spate același lucru: un **integrator numeric** care avansează starea cu pași de timp

! Atenție la setări

Vensim folosește implicit metoda Euler cu un pas fix. Ora 2 arată când acest lucru poate produce rezultate greșite – și cum verificăm.

Verificare

? Numărul de utilizatori ai unui serviciu crește cu **7% pe lună**, continuu. În aproximativ cât timp se dublează?

A 7 luni

B 10 luni

C 14 luni

D 70 de luni

De ce avem nevoie de metode numerice

- Exponențiala, logistica și răcirea au **formulă**: le-am putut desena exact
- Modelul Bass are și el formulă, dar orice extensie (abandon, preț variabil) o pierde
- Lotka–Volterra, SIR, World3, vremea: **nicio formulă** – doar ecuația și starea inițială
- Soluția: calculăm starea **pas cu pas**, cum am făcut în codul stoc–flux
- Întrebările orei 2: cât de mare poate fi pasul? cât de mare este eroarea? ce se întâmplă dacă pasul e prea mare?

T **Teaser**

Aceeași ecuație Lotka–Volterra, același pas de timp: cu Euler populațiile explodează, cu Runge–Kutta 4 ciclurile rămân închise. Doar metoda diferă.

Rezumatul orei 1

- ✓ Model dinamic: **stare**, intrări, ieșiri, parametri; ecuație de bilanț: creare − distrugere
- ✓ Orice EDO se reduce la **s’ = f(s, t)** de ordinul 1 – un singur integrator rezolvă tot
- ✓ Exponențială ($T_2 = \ln 2/r$), **logistică** (echilibre 0 instabil, C stabil); harta logistică discretă poate fi **haotică**
- ✓ Răcirea lui Newton: 90 → 60 °C în ~7 min; k se estimează din date (prima calibrare)
- ✓ Stocuri și fluxuri, bucle R și B, **întârzieri → oscilații**; modelul Bass pentru adoptarea aplicațiilor

Ora 2

Integrare numerică

1. Metoda Euler: eroare locală și globală
2. Heun și Runge–Kutta 4: ordinul unei metode
3. Stabilitate, sisteme rigide, metode implicite
4. Pas adaptiv (solve_ivp) și demo-ul Lotka–Volterra

Problema: de la rată la traiectorie

- Știm $\mathbf{s(t_0) = s_0}$ și rata $\mathbf{s' = f(t, s)}$; vrem s(t) la momentele $t_0 + h$, $t_0 + 2h$, …
- Formal: $s(t + h) = s(t) + \int f(\tau, s(\tau))\, d\tau$ pe intervalul $[t, t + h]$ – dar integrala depinde de s, pe care nu-l știm
- Toate metodele **aproximează această integrală** folosind valori ale lui f
- **Pasul h**: mic = precis, dar multe evaluări; mare = rapid, dar eroare mare sau chiar instabilitate
- Costul se măsoară în **evaluări ale lui f** – la modele mari, fiecare evaluare poate fi scumpă

I

Legătura cu S2

Integrarea numerică a unei EDO seamănă cu integrarea unei funcții (dreptunghiuri, trapeze, Simpson) – doar că funcția de integrat depinde de soluția însăși.

Metoda Euler (1768)

$$s(n+1) = s(n) + h \cdot f(t(n), s(n))$$

geometric	mergem un pas h pe **tangenta** la soluție în punctul curent (metoda liniilor poligonale)
Taylor	$s(t + h) = s(t) + h \cdot s'(t) + h^2/2 \cdot s''(t) + \ldots$: Euler păstrează doar primii doi termeni
cost	**o singură** evaluare a lui f pe pas; precizie minimă (Holban: „efort minim, precizie minimă”)

Leonhard Euler a descris metoda în Institutionum calculi integralis (1768–1770). Este metoda implicită a multor unelte de dinamica sistemelor și pasul de bază din care derivă aproape toate celelalte.

Un pas Euler de mână: cafeaua, h = 1 minut

t	Calculul Euler	Euler	Exact	Eroare
0	condiția inițială	90	90	0
1	$90 + 1 \cdot (-0{,}08 \cdot (90 - 20)) = 90 - 5{,}6$	84,40	84,62	0,22
2	$84{,}4 - 0{,}08 \cdot 64{,}4 = 84{,}4 - 5{,}152$	79,25	79,65	0,40
3	$79{,}248 - 0{,}08 \cdot 59{,}248 = 79{,}248 - 4{,}740$	74,51	75,06	0,55

- Euler subestimează constant temperatura: folosește panta de la **începutul** pasului, când cafeaua se răcea cel mai repede
- Eroarea crește de la un pas la altul – se acumulează: aceasta este **eroarea globală**

Euler în Python (pentru orice sistem)

```
import numpy as np

def euler(f, t0, s0, h, n):
    t, s = t0, np.array(s0, float)
    T, S = [t], [s.copy()]
    for _ in range(n):
        s = s + h * f(t, s)     # un pas pe tangentă
        t += h
        T.append(t); S.append(s.copy())
    return np.array(T), np.array(S)

f = lambda t, s: -0.08 * (s - 20)   # cafeaua
T, S = euler(f, 0, [90], 1.0, 10)
```

- s poate fi un **vector** (sisteme)
- f primește (t, s), ca în SciPy
- ~10 linii, aceeași idee ca în modelul stoc–flux
- Vom scrie la fel Heun și RK4

Exemplul din Holban: $y' = 2xy$, $y(1) = 1$, Euler cu $h = 0{,}1$

x	y Euler	exact e^(x² – 1)	eroarea	eroarea %
1,1	1,2000	1,2337	0,0337	2,73
1,2	1,4640	1,5527	0,0887	5,71
1,3	1,8154	1,9937	0,1784	8,95
1,4	2,2874	2,6117	0,3243	12,42
1,5	2,9278	3,4903	0,5625	16,12

- La x = 1,5: eroare de **16,1%** cu h = 0,1; 9,1% cu h = 0,05; 2,0% cu h = 0,01
- Pasul de 10 ori mai mic reduce eroarea de ~8 ori: aproape **proporțional cu h** – semnul unei metode de ordinul 1

Eroarea locală și eroarea globală

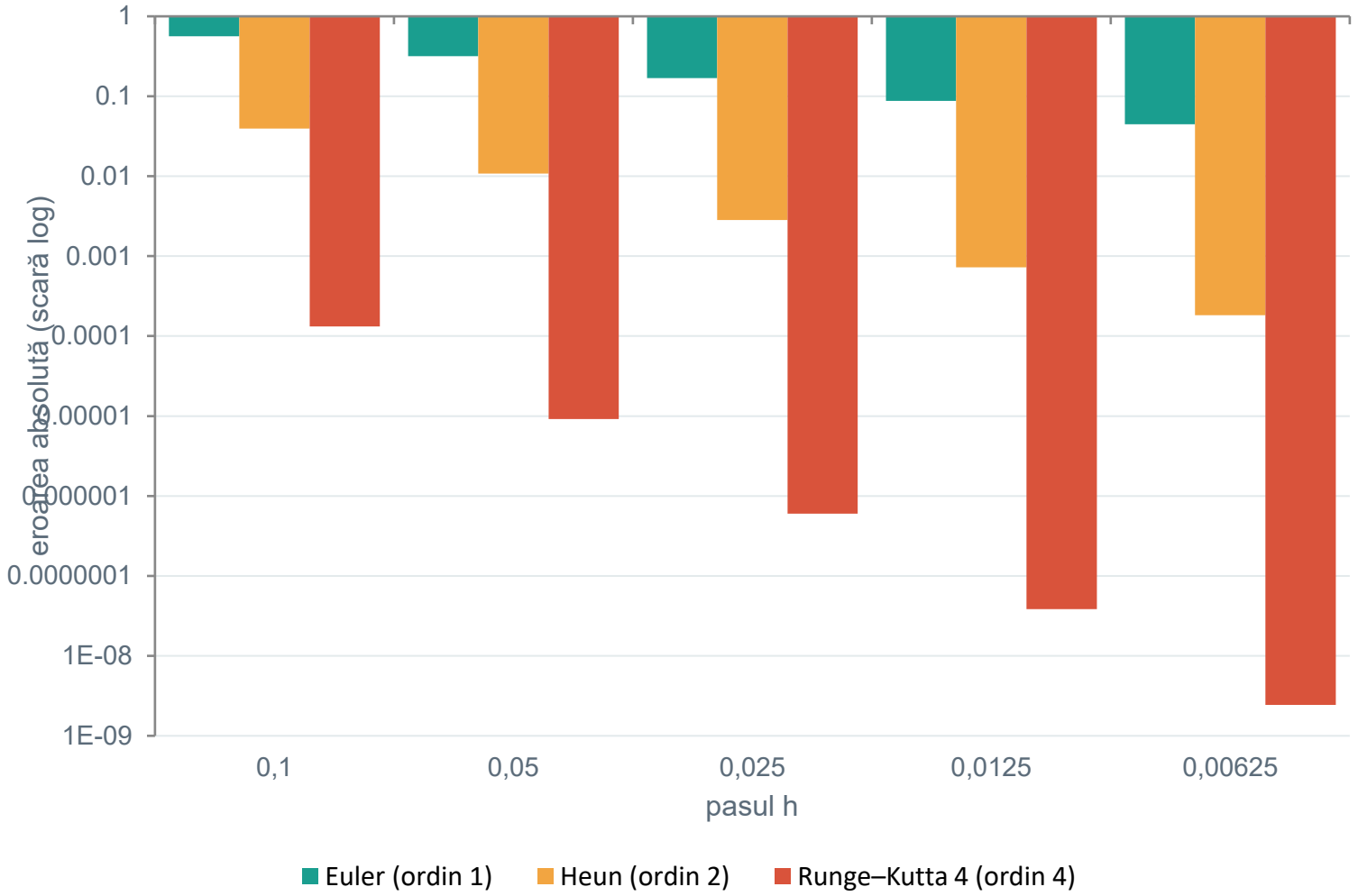
Locală $O(h^2)$, globală $O(h)$

Eroarea **locală** este greșeala făcută într-un singur pas, pornind dintr-un punct exact: pentru Euler, termenul Taylor ignorat, ~$h^2/2 \cdot s''$. Eroarea **globală** este diferența finală, după toți cei $(T - t_0)/h$ pași: erorile locale se adună și se propagă, deci ~$h^2 \cdot (1/h) = h$.

Holban cap. 6; Hairer, Nørsett, Wanner, 1993

- O metodă are **ordinul p** dacă eroarea globală scade ca h^p: Euler p = 1
- Înjumătățind pasul, eroarea Euler se înjumătățește (dar costul se dublează)
- Erorile de rotunjire (virgula mobilă) cresc când h este foarte mic – există un pas optim, dar în practică rar îl atingem

Ordinul de convergență: eroarea la x = 1,5 vs pasul h



- Pas înjumătățit: Euler ÷ 1,96, Heun ÷ 3,96, RK4 ÷ 15,8
- Teoretic: ÷ 2, ÷ 4, ÷ 16
- RK4 cu h = 0,1: $1{,}3 \cdot 10^{-4}$
- Euler cu h = 0,00625: $4{,}5 \cdot 10^{-2}$

Metoda Heun (Euler îmbunătățit, predictor–corector)

$$k_1 = f(t_n, s_n), \quad k_2 = f(t_n + h, s_n + h \cdot k_1), \quad s_{n+1} = s_n + h \cdot (k_1 + k_2)/2$$

predictor	un pas Euler „de probă” până la $t_n + h$, ca să aflăm panta la **sfârșitul** pasului
corector	mergem pe **media** celor două pante – regula trapezului pentru integrală
rezultat	ordinul 2, cu 2 evaluări pe pas; exemplul Holban cu h = 0,1: eroare **1,13%** (Euler: 16,1%)

Valorile Heun din curs (3,4509 / 3,4795 / 3,4899 la x = 1,5) sunt corecte – le-am reprodus exact. Pentru cafea, Heun corectează aproape complet subestimarea lui Euler.

Runge–Kutta de ordinul 4 (1895–1901)

$$s_{n+1} = s_n + h/6 \cdot (k_1 + 2k_2 + 2k_3 + k_4)$$

k_1, k_2	$k_1 = f(t_n, s_n)$ la început; $k_2 = f(t_n + h/2, s_n + h/2 \cdot k_1)$ la mijloc
k_3, k_4	$k_3 = f(t_n + h/2, s_n + h/2 \cdot k_2)$ din nou la mijloc; $k_4 = f(t_n + h, s_n + h \cdot k_3)$ la sfârșit
ponderile 1-2-2-1	media ponderată a pantelor seamănă cu regula lui **Simpson**; eroarea globală ~ $\mathbf{h^4}$

Carl Runge (1895) și Wilhelm Kutta (1901). Cu 4 evaluări pe pas, RK4 este „calul de povară” al simulării: precizie excelentă, cod scurt, fără derivate de ordin superior.

RK4 în Python

```
def rk4(f, t0, s0, h, n):
    t, s = t0, np.array(s0, float)
    T, S = [t], [s.copy()]
    for _ in range(n):
        k1 = f(t, s)
        k2 = f(t + h/2, s + h/2 * k1)
        k3 = f(t + h/2, s + h/2 * k2)
        k4 = f(t + h, s + h * k3)
        s = s + h/6 * (k1 + 2*k2 + 2*k3 + k4)
        t += h
        T.append(t); S.append(s.copy())
    return np.array(T), np.array(S)
```

- Aceeași interfață ca euler()
- 4 evaluări ale lui f pe pas
- Funcționează pentru vectori
- Verificați ordinul: pas ÷ 2 → eroare ÷ 16

Precizie vs cost: eroarea la x = 1,5

Metoda	h = 0,1	h = 0,05	h = 0,01
Euler	0,5625 (5 ev.)	0,3171 (10 ev.)	0,0706 (50 ev.)
Heun	0,0394 (10 ev.)	0,0108 (20 ev.)	0,000464 (100 ev.)
Runge–Kutta 4	0,000132 (20 ev.)	$9{,}1 \cdot 10^{-6}$ (40 ev.)	$1{,}6 \cdot 10^{-8}$ (200 ev.)

- Cu **20** de evaluări, RK4 este de ~500 de ori mai precis decât Euler cu **50** de evaluări
- Regula: la precizie egală, metodele de ordin mare sunt aproape întotdeauna mai ieftine – de aceea RK4 și RK45 sunt standardul

Metode cu un pas și metode cu mai mulți pași (Holban 6.3)

Cu un pas (Runge–Kutta)

- folosesc doar starea curentă s_n
- evaluează f de mai multe ori **în interiorul** pasului
- pornesc singure și își schimbă ușor pasul
- Euler, Heun, RK4, RK45

Cu mai mulți pași (Adams, BDF)

- refolosesc pantele din pașii **anteriori**: f_n, f_{n-1}, …
- Adams–Bashforth 2: $s_{n+1} = s_n + h\cdot(3/2\cdot f_n - 1/2\cdot f_{n-1})$
- o singură evaluare nouă pe pas: ieftine
- au nevoie de alt start; schimbarea pasului e mai complicată

Holban deduce Adams–Bashforth 2 integrând dreapta care trece prin (x_{n-1}, f_{n-1}) și (x_n, f_n). Metodele BDF, cu mai mulți pași și implicite, sunt baza solverelor pentru sisteme rigide (BDF și LSODA în SciPy).

Erori în tabelele Holban (cap. 6), găsite prin recalculare

Unde	În curs	Corect
Exemplul Lotka–Volterra 1	$\alpha = 0{,}2$ în text	$\alpha = 2$ (ecuația $2x - 1{,}1xy$ și echilibrul 10/9, 20/11)
Euler, h = 0,05, coloana %	1,00; 1,10; … (copiată din x)	1,47; 3,11; 4,93; 6,92; **9,08**
RK4, h = 0,1, la x = 1,5	3,4903429 (eroare 0,000057)	3,490211 (eroare 0,000132)
RK4, h = 0,01	identic cu h = 0,1	eroare $\approx 1{,}6 \cdot 10^{-8}$ (de ~8 400 de ori mai mică)
Taylor cu trei termeni	$y_{n+1} = y_n + y'_n h + y''_n h^2$	lipsește **împărțirea la 2**: $y''_n h^2/2$

- Verificarea ordinului (pas ÷ 2 → eroare ÷ 2^p) ar fi semnalat imediat tabelele RK4: eroarea nu poate crește când pasul scade
- Valorile Euler și Heun din curs sunt corecte; doar coloana de procente și tabelele RK4 au fost transcrise greșit

Verificare

? Pentru $y' = -2y$, $y(0) = 1$, cu pasul $h = 0{,}1$, cât este y după **un pas Euler**?

A 0,90

B 0,80

C 0,82

D 1,20

Când pasul e prea mare

Stabilitate și sisteme rigide

1. Ecuația de test $y' = -\lambda y$
2. Pragul de stabilitate al metodei Euler: $h < 2/\lambda$
3. Sisteme rigide (stiff) și metode implicite
4. Ce aleg solverele moderne

Ecuația de test și pragul de stabilitate

$$y' = -\lambda \cdot y \implies \text{Euler: } y_{n+1} = (1 - h \cdot \lambda) \cdot y_n$$

exact	y(t) = e^(−λt) scade monoton spre 0, pentru orice λ > 0
Euler	$y_n = (1 - h\lambda)^n$: scade doar dacă $\lvert 1 - h\lambda \rvert < 1$, adică **h < 2/λ**
exemplu	λ = 10: h < 0,2; cu h = 0,25 factorul este −1,5 și soluția **explodează** cu semne alternante

Pentru 1/λ < h < 2/λ soluția este stabilă, dar oscilează (factor negativ). Pragul nu ține de precizie, ci de **stabilitate**: chiar dacă nu ne interesează detaliile rapide, pasul trebuie să le respecte.

$y' = -10 \cdot y$ cu Euler: trei pași de timp

Pasul h (factorul 1 – 10h)	n = 0	1	2	3	4	5	6
h = 0,05 (0,5)	1,0000	0,5000	0,2500	0,1250	0,0625	0,0312	0,0156
h = 0,15 (−0,5)	1,0000	-0,5000	0,2500	-0,1250	0,0625	-0,0312	0,0156
h = 0,25 (−1,5)	1,000	-1,500	2,250	-3,375	5,063	-7,594	11,391
exact la t = n·0,05	1,0000	0,6065	0,3679	0,2231	0,1353	0,0821	0,0498

- h = 0,05: scade lin; h = 0,15: scade, dar **oscilează** între semne; h = 0,25: crește fără limită – după 2 unități de timp ajunge la ±25
- Soluția exactă este practic 0 după t = 0,5: o metodă instabilă transformă un sistem liniștit într-o explozie numerică

Sisteme rigide (stiff)

Scări de timp foarte diferite

Un sistem este **rigid** când conține procese foarte rapide, care se sting repede, alături de procese lente care ne interesează. Metodele explicite sunt obligate să folosească pasul impus de procesul **cel mai rapid**, deși acesta nu mai contează după primele momente.

Curtiss & Hirschfelder, 1952; Hairer & Wanner, 1996

- Exemplu: y′ = −50·(y − cos t): y urmărește lent cos t, dar orice abatere dispare în ~0,02 unități
- Euler explicit cere h < 2/50 = **0,04** pe tot intervalul, deși cos t s-ar putea urmări cu h = 0,5
- Unde apar: cinetică chimică, circuite electronice, modele climatice, rețele neuronale biologice

Metoda Euler implicită (backward Euler)

$$s_{n+1} = s_n + h \cdot f(t_{n+1}, s_{n+1})$$

diferența	panta se ia la **sfârșitul** pasului, în punctul necunoscut: trebuie rezolvată o ecuație la fiecare pas
ecuația de test	$y_{n+1} = y_n/(1 + h\lambda)$: factorul este mereu sub 1, deci **stabil pentru orice h** (A-stabil)
costul	un sistem de ecuații (de obicei cu metoda Newton și matricea Jacobi) la fiecare pas

Pentru exemplul rigid, ecuația implicită are soluție directă: $y_{n+1} = (y_n + 50h \cdot \cos t_{n+1})/(1 + 50h)$. Solverele implicite profesionale: BDF (Gear, 1971), Radau, LSODA (alege automat între explicit și implicit).

Exemplul rigid $y' = -50\cdot(y - \cos t)$ în cifre

$7{,}4\cdot10^{6}$

$|y|$ atins de Euler **explicit** cu h = 0,05 în doar 2 unități de timp (factor −1,5 pe pas)

0,00043

eroarea Euler **implicit** cu același pas h = 0,05, la t = 10

2 942 vs 749

evaluări ale lui f: solve_ivp cu **RK45** (explicit) vs **Radau** (implicit), aceeași toleranță

- RK45 este stabil doar pentru că își micșorează singur pasul: 487 de pași față de 105 pentru Radau
- Semnul unui sistem rigid: solverul explicit face mulți pași mici chiar și unde soluția este netedă

Unelte profesionale

Pas adaptiv și solve_ivp

1. Ideea: estimăm eroarea și ajustăm pasul
2. Perechi încorporate: RK45 (Dormand–Prince)
3. solve_ivp: metode, toleranțe, evenimente
4. Simulare hibridă: evenimente în modele continue

Cum funcționează pasul adaptiv

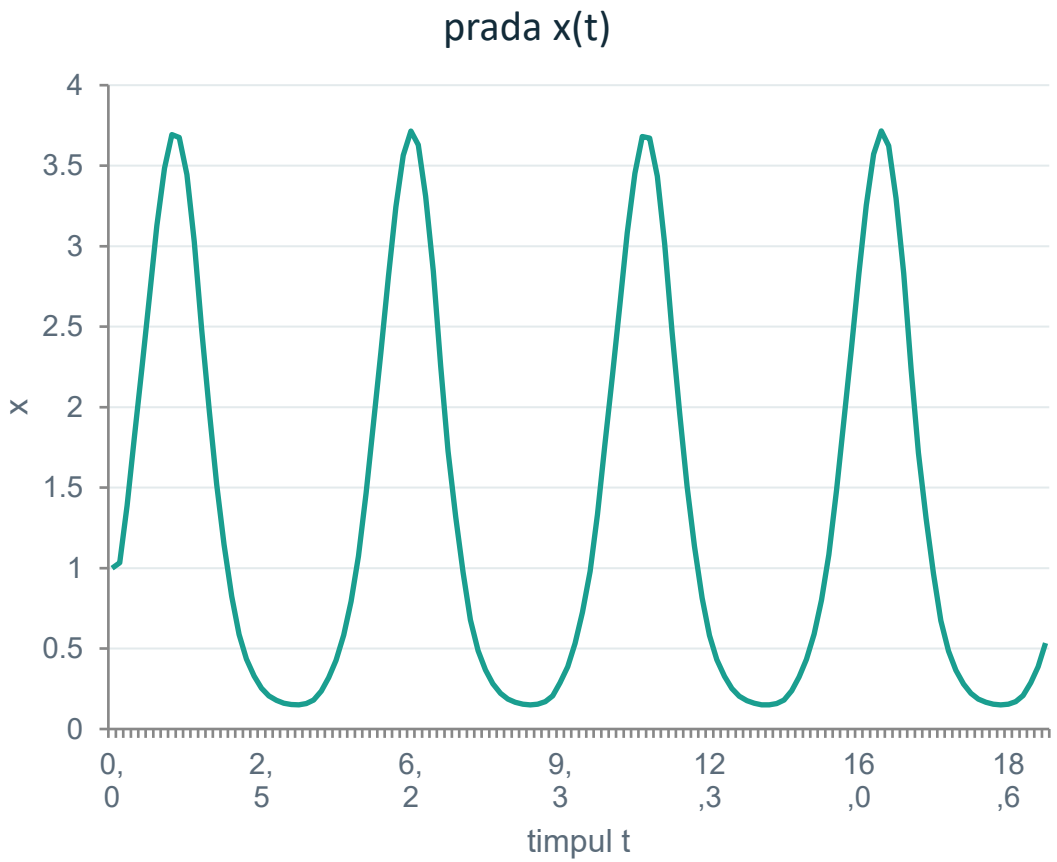
- Calculăm pasul cu **două metode** de ordine diferite (de ex. 4 și 5), care împart aceleași evaluări ale lui f
- Diferența dintre ele este o **estimare a erorii locale**, gratuită
- Dacă eroarea > toleranța: respingem pasul și îl reluăm cu h mai mic; altfel îl acceptăm și eventual mărim h
- Toleranța: $|\text{eroare}| \le \text{atol} + \text{rtol}\cdot|s|$ – **rtol** (relativă, implicit 10^{-3}) și **atol** (absolută, implicit 10^{-6})
- Rezultat: pași mici unde soluția se schimbă repede, pași mari unde este netedă

R

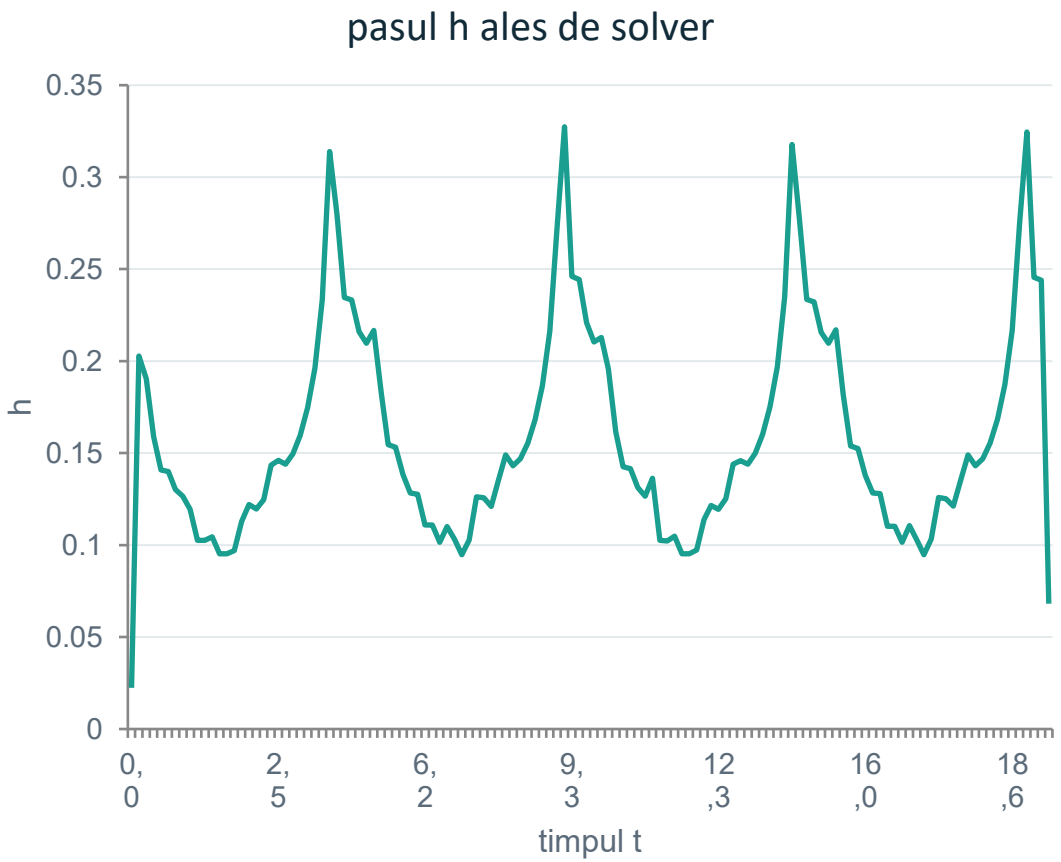
RK45

Dormand și Prince (1980): 7 evaluări pe pas (6 efective, ultima se refolosește), ordinul 5 cu estimare de ordinul 4. Este metoda implicită în solve_ivp și în ode45 din MATLAB.

Pasul adaptiv pe Lotka–Volterra (RK45, rtol = 10^{-6})



vârfuri ascuțite de pradă



h între **0,022** și **0,33**

solve_ivp: modul corect de a integra în Python

```
from scipy.integrate import solve_ivp
import numpy as np

def lv(t, z, a=2.0, b=1.1, c=1.0, d=0.9):
    x, y = z
    return [a*x - b*x*y, d*x*y - c*y]

sol = solve_ivp(lv, [0, 30], [1.0, 0.5],
                method="RK45",       # sau "Radau", "LSODA"
                rtol=1e-6, atol=1e-9,
                t_eval=np.linspace(0, 30, 301))

print(sol.success, sol.nfev)    # True, numărul de evaluări
x, y = sol.y                    # traiectoriile
```

- f(t, z) întoarce derivatele
- t_eval: unde vrem rezultate (nu pașii interni)
- Verificați **sol.success**
- Schimbați rtol: rezultatul nu trebuie să se schimbe vizibil

Alegerea metodei și evenimentele

- **RK45**: implicit, pentru probleme nerigide; **DOP853**: precizie foarte mare (ordin 8)
- **Radau**, **BDF**: probleme rigide; **LSODA**: detectează singur rigiditatea și comută între metode
- **Evenimente**: solve_ivp(…, events=g) găsește momentele în care g(t, s) = 0 – o populație scade sub un prag, o bilă atinge podeaua
- Un eveniment poate **opri** integrarea (terminal=True); modificăm starea și repornim: simulare **hibridă**
- Exemplu: termostatul – temperatura evoluează continuu, iar pornirea și oprirea încălzirii sunt evenimente (S5 + S7)

Regula de aur

Nu alegeți pasul „după ochi”. Folosiți un solver adaptiv, verificați rezultatul cu o toleranță de 10 ori mai mică și treceți la o metodă implicită dacă solverul devine neobișnuit de lent.

Demo-ul săptămânii

Lotka–Volterra: pradă și prădător (Holban 6.2)

1. Ecuațiile și echilibrul
2. Euler vs RK4 cu același pas de timp
3. O mărime conservată: testul de verificare
4. Modelul extins și principiul lui Volterra

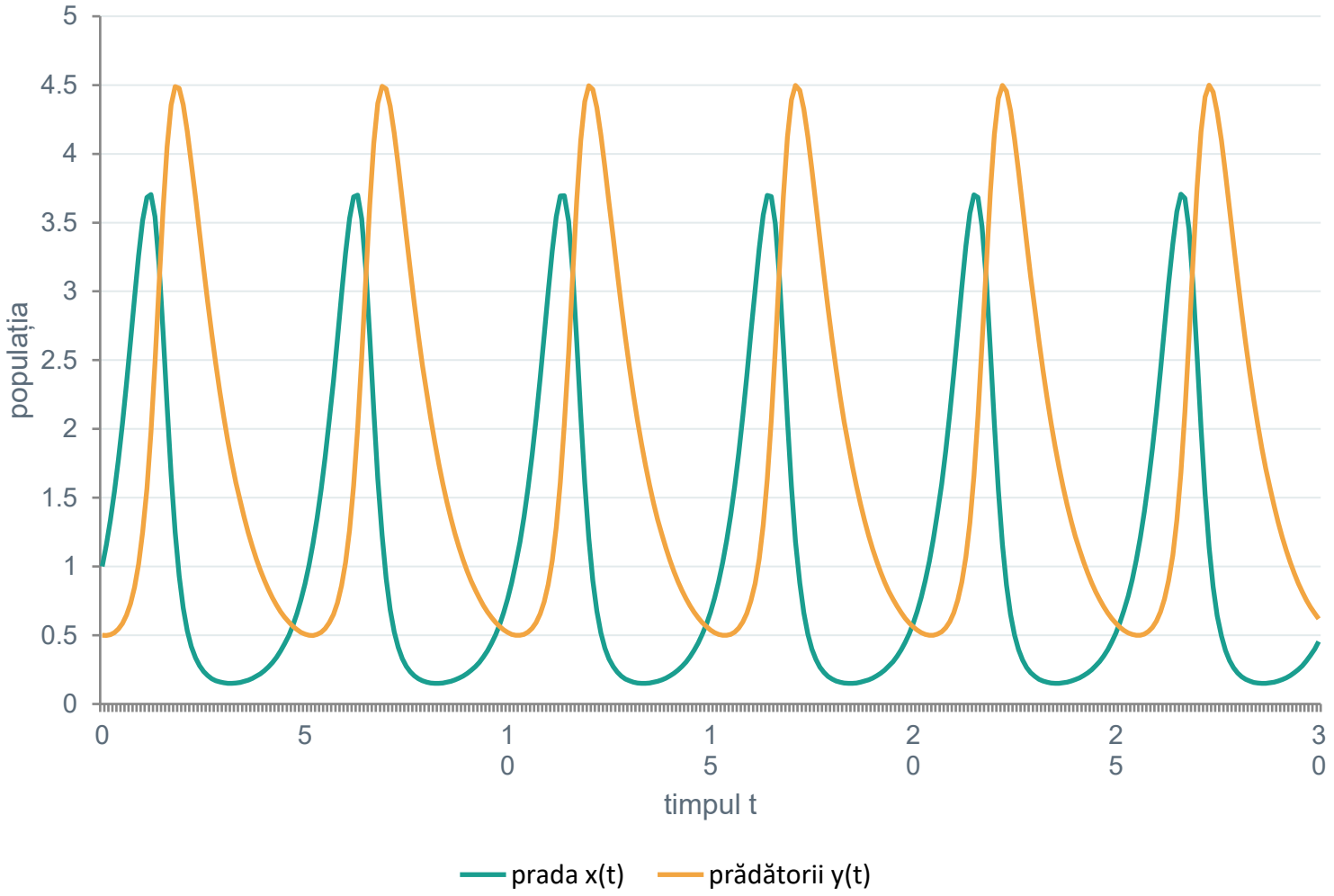
Modelul Lotka–Volterra (1925–1926)

$$dx/dt = \alpha \cdot x - \beta \cdot x \cdot y, \qquad dy/dt = \delta \cdot x \cdot y - \gamma \cdot y$$

x, y	prada (iepuri) și prădătorii (vulpi, râși); α creșterea prăzii, γ moartea prădătorilor
βxy, δxy	întâlnirile pradă–prădător: prada scade, prădătorii cresc (aceeași logică de contact ca la SIR)
exemplul 1	α = 2, β = 1,1, γ = 1, δ = 0,9, x(0) = 1, y(0) = 0,5; echilibru (γ/δ, α/β) = (10/9, 20/11) ≈ (1,11; 1,82)

Presupuneri (Holban): prada are hrană nelimitată, prădătorii depind doar de această pradă, mediul nu se schimbă. Nerealist în detaliu, dar explică ciclurile observate timp de un secol în datele Hudson’s Bay Company.

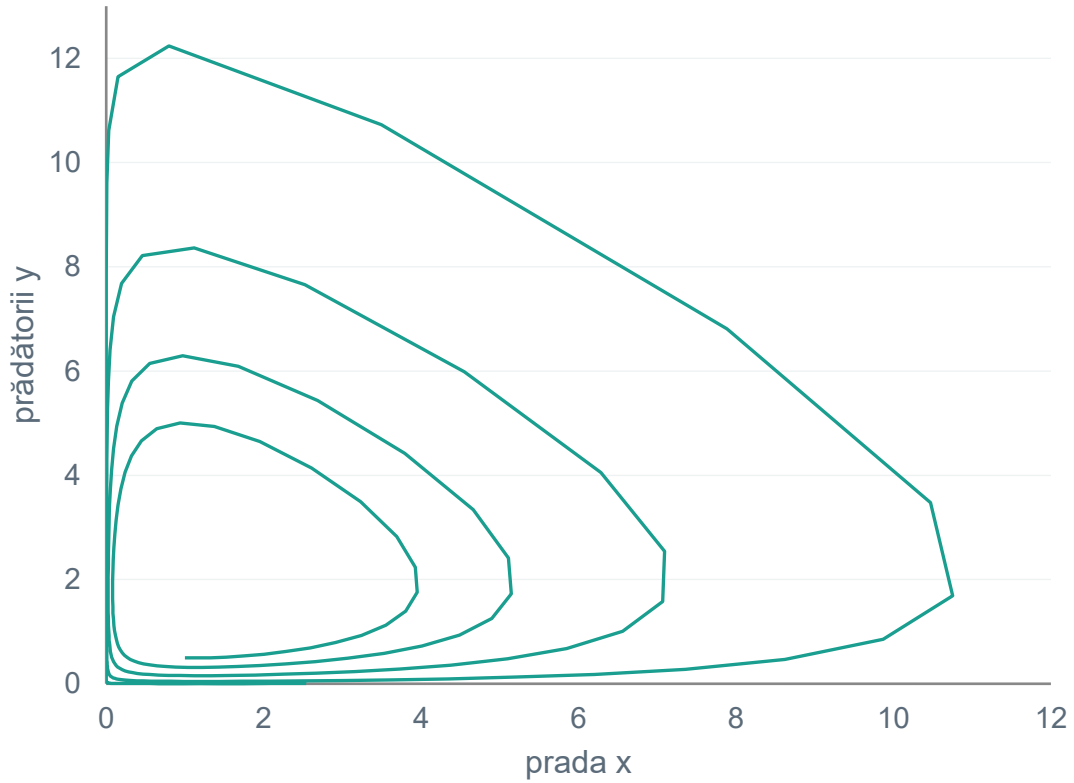
Soluția exactă (solver cu toleranță 10^{-10}): cicluri regulate



- Prada oscilează între **0,15** și **3,71**
- Perioada: **≈ 5,1** unități de timp
- Vârful prădătorilor vine **după** vârful prăzii
- Ciclurile se repetă identic, la nesfârșit

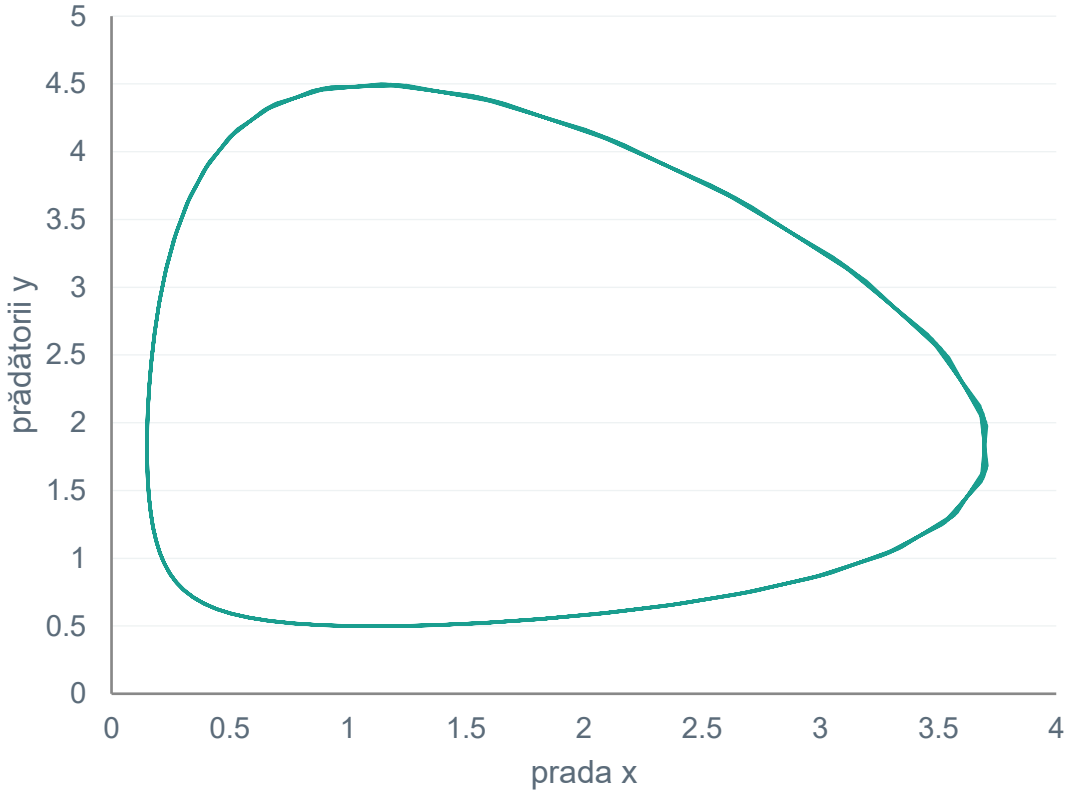
Același pas h = 0,05, 30 de unități de timp: Euler vs Runge–Kutta 4

Euler



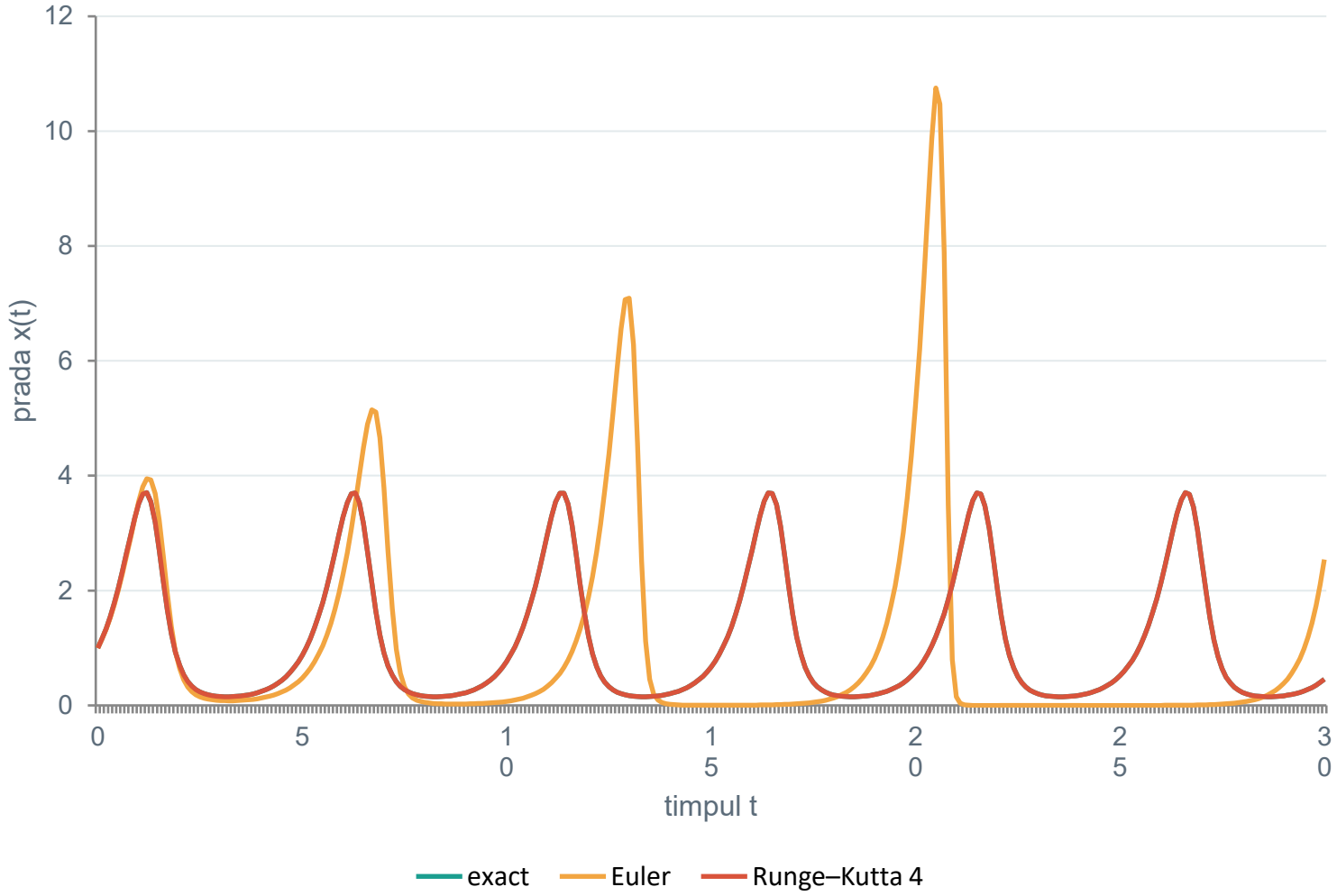
spirala „explodează”: x ajunge la **10,8**

Runge–Kutta 4



orbita rămâne **închisă** (x max 3,72)

Prada în timp: exact vs Euler vs RK4 (h = 0,05)



- RK4 se suprapune peste soluția exactă
- Euler: vârfuri tot mai înalte și mai rare
- Între vârfuri, prada Euler scade aproape la **0**
- Un ecolog ar concluziona greșit că sistemul se destabilizează

Verificarea cu o mărime conservată

Metoda (30 de unități de timp)	Evaluări f	V la final	x maxim
Valoarea inițială V(1; 0,5)	–	2,8363	3,707 (exact)
Euler, h = 0,05	600	**12,540**	10,83
Euler, h = 0,005	6 000	3,208	4,22
Runge–Kutta 4, h = 0,05	2 400	**2,8363**	3,716
Euler simplectic, h = 0,05	600	2,842	3,728

- $V(x, y) = \delta x - \gamma \cdot \ln x + \beta y - \alpha \cdot \ln y$ rămâne **constant** pe orice soluție exactă: orbitele sunt curbele de nivel ale lui V
- Chiar cu un pas de 10 ori mai mic (6 000 de evaluări), Euler deviază; RK4 cu 2 400 de evaluări păstrează V la a patra zecimală

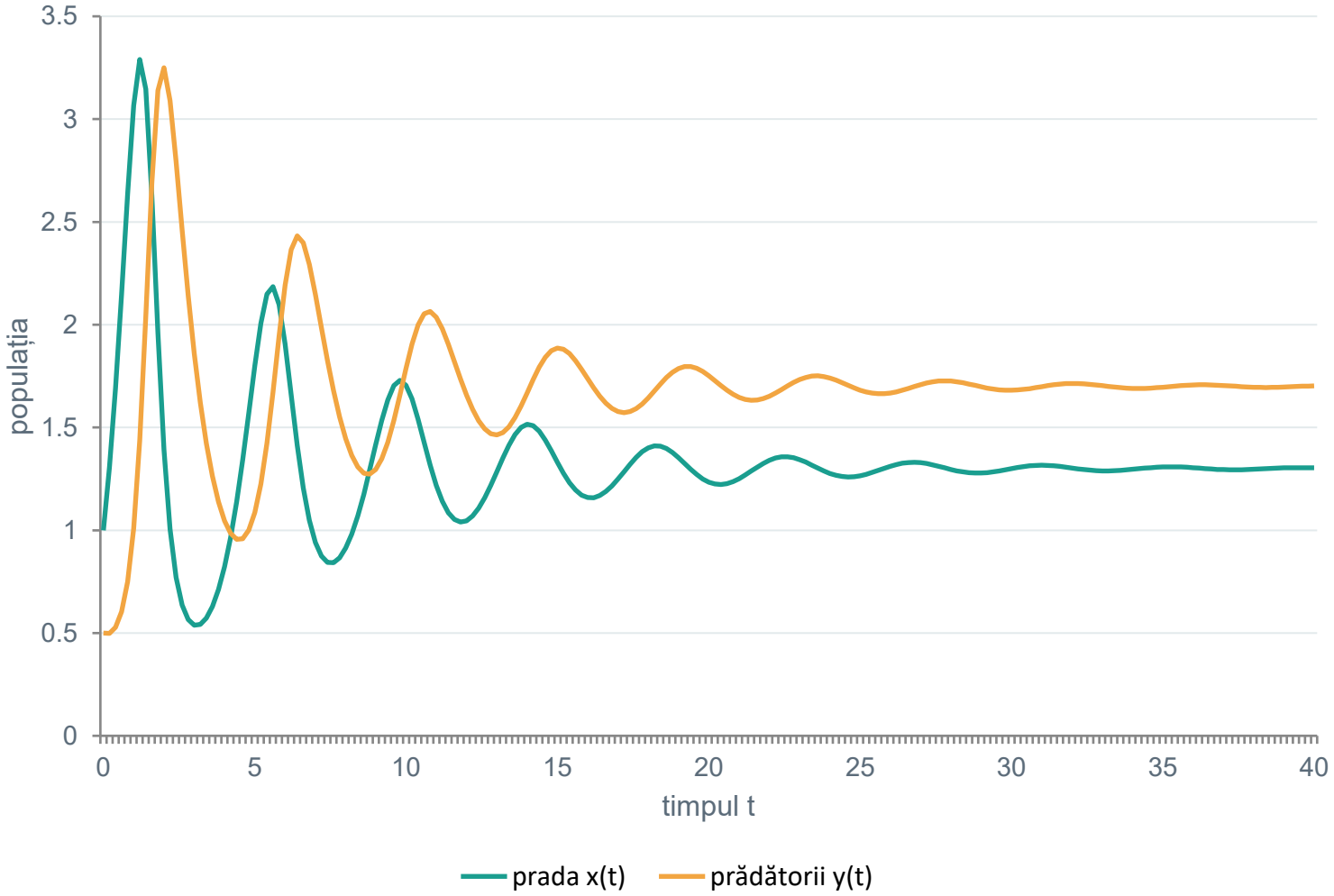
De ce explodează Euler și ce învățăm

- Orbitele sunt curbe **închise și convexe**; pasul pe tangentă iese mereu puțin **în afara** curbei
- Fiecare pas mută traiectoria pe o orbită „mai mare”: erorile nu se compensează, ci se **adună** în aceeași direcție
- **Euler simplectic**: actualizăm x, apoi calculăm y cu x **nou** – o mică schimbare care păstrează structura (orbitele rămân închise)
- Lecția de verificare (S6): testați codul cu **mărimi conservate** (energie, masă, V) și cu pași diferiți
- Un rezultat calitativ greșit (populații care explodează) poate părea plauzibil dacă nu știm ce să căutăm

V **Pentru S8**

Metodele simplectice (Verlet, leapfrog) sunt standardul pentru simulări de particule și astronomie: păstrează energia pe milioane de pași.

Modelul extins (Holban 6.2.2): competiție în interiorul speciilor



- dx/dt = 2x – **0,1x²** – 1,1xy
- dy/dt = 0,9xy – **0,1y²** – y
- Oscilațiile se **amortizează**: echilibru (1,30; 1,70)
- Valoarea din curs, (1,3; 1,7), este confirmată

Principiul lui Volterra: o poveste de pescuit

- Umberto D'Ancona observă că în Primul Război Mondial, când pescuitul în Adriatica a scăzut, **proporția rechinilor** în capturi a crescut
- Volterra (1926) explică prin model: pescuitul scade α (prada) și crește γ (prădătorii)
- Echilibrul (γ/δ, α/β): **mai puțin pescuit → mai puțină pradă și mai mulți prădători** la echilibru
- Consecință contraintuitivă: un insecticid care ucide și dăunătorii, și prădătorii lor poate **crește** numărul de dăunători
- Media pe un ciclu a lui x este exact γ/δ – perioada pentru oscilații mici: $2\pi/\sqrt{(\alpha\gamma)} \approx 4{,}44$ (aici 5,1, oscilații mari)

E **Model ca explicație**

Lotka–Volterra nu prezice exact populațiile, dar explică **de ce** apar cicluri și cum răspunde sistemul la o intervenție – valoarea lui principală.

Același fenomen la un arc: $x'' = -x$, 10 perioade, $h = 0{,}1$

Energia $E = (x^2 + v^2)/2$ la final

- la start: **0,5**
- Euler: **258,7** (de ~500 de ori mai mare)
- RK4: **0,499996** (pierde foarte puțin)
- Euler simplectic: **0,5003** (oscilează în jurul lui 0,5)

Ce înseamnă

- Euler înmulțește energia cu $(1 + h^2)$ la fiecare pas
- RK4 disipă foarte puțin, dar sistematic
- Simplecticul nu e exact, dar **nu derivă** pe termen lung
- pentru orbite, molecule, jocuri: metode simplectice

Reducerea la ordinul 1 din ora 1 în acțiune: x'' = −x devine x' = v, v' = −x și se rezolvă cu aceleași funcții euler() și rk4(). Simulări de particule cu milioane de pași – în S8.

Verificare

? Rezolvăm Lotka–Volterra cu Euler și pasul h = 0,05, iar populațiile cresc de la un ciclu la altul. Ce concluzie este corectă?

A ecosistemul modelat este instabil

B parametrii α și β sunt greșiți

C este o eroare numerică: metoda nu păstrează mărimea conservată V

D trebuie mai multe replicări

Lista de verificare pentru integrarea numerică

1 Soluție cunoscută

Testați codul pe o problemă cu formulă (răcirea, exponențiala) și verificați **ordinul**.

2 Pas sau toleranță ÷ 10

Rezultatul nu trebuie să se schimbe vizibil; dacă se schimbă, nu era corect.

3 Mărimi conservate

Energie, masă, populație totală (S + I + R = N), V la Lotka–Volterra.

4 Valori imposibile

Populații negative, probabilități peste 1, oscilații care nu există în realitate.

5 Rigiditate

Solverul explicit e lent sau face pași minusculi? Încercați Radau, BDF sau LSODA.

6 Documentare

Metoda, pasul sau toleranțele, versiunea SciPy – ca orice rezultat reproductibil (S6).

Rezumatul orei 2

- ✓ **Euler**: un pas pe tangentă; eroare locală $O(h^2)$, globală $O(h)$ – ordinul 1
- ✓ **Heun** (ordin 2) și **Runge–Kutta 4** (ordin 4): pas ÷ 2 → eroare ÷ 4, respectiv ÷ 16
- ✓ **Stabilitate**: Euler pe $y' = -\lambda y$ cere $h < 2/\lambda$; sisteme **rigide** → metode implicite (Radau, BDF, LSODA)
- ✓ **solve_ivp**: pas adaptiv (RK45), toleranțe rtol/atol, evenimente pentru simulări hibride
- ✓ Lotka–Volterra: cu același pas, Euler **explodează**, RK4 păstrează orbitele; verificăm cu mărimea conservată V

Ora 3

Epidemii și calibrare

1. Modelul SIR (Kermack–McKendrick, 1927) și R_0
2. Intervenții, SEIR, epidemii stocastice
3. Lecții din COVID-19: modele în decizii publice
4. Calibrarea pe date reale: gripa dintr-un internat, iepuri și râși

De ce modele pentru epidemii

- **1760**: Daniel Bernoulli calculează câți ani de viață câștigă populația prin variolizare – primul model epidemiologic
- **1911**: Ronald Ross arată matematic că malaria poate fi oprită fără a ucide toți țânțarii (un prag)
- **1927**: Kermack și McKendrick publică modelul SIR, folosit pentru ciuma din Bombay (1906)
- Azi: planificarea vaccinărilor, capacitatea spitalelor, scenarii pentru gripă, rujeolă, COVID-19
- În informatică: virusuri și viermi informatici, protocoale gossip, răspândirea informației (și a dezinformării) în rețele

P **Ideea centrală**

O epidemie nu are nevoie să infecteze pe toată lumea ca să se oprească: există un **prag**, iar modelul spune unde este.

Modelul SIR: trei compartimente



Infectarea: rata β·S·I/N
proporțională cu numărul de întâlniri dintre susceptibili și infectați (amestec omogen)

Vindecarea: rata γ·I
fiecare infectat se vindecă în medie după 1/γ zile; N = S + I + R rămâne constant

Ecuațiile SIR (fracții din populație)

$$s' = -\beta \cdot s \cdot i, \quad i' = \beta \cdot s \cdot i - \gamma \cdot i, \quad r' = \gamma \cdot i$$

s, i, r	fracțiile de susceptibili, infectați, vindecați; s + i + r = 1 (o ecuație este redundantă)
β	contacte pe zi × probabilitatea de transmitere la un contact
presupuneri	populație fixă și **omogen amestecată**, fără nașteri, imunitate permanentă, fără incubație (Holban cap. 7)

Cu numere absolute (S, I, R) termenul de infectare devine β·S·I/N. Modelul nu are formulă pentru s(t) și i(t): îl rezolvăm cu solve_ivp, ca pe Lotka–Volterra.

Numărul de reproducere de bază

$$R_0 = \beta / \gamma$$

sens	numărul mediu de persoane infectate de **un** bolnav într-o populație complet susceptibilă
compunere	R_0 = contacte pe zi × probabilitatea de transmitere × durata bolii ($1/\gamma$ zile)
exemplu	$\beta = 0{,}5$/zi, boala durează $1/\gamma = 5$ zile → $R_0 = 0{,}5 \cdot 5 =$ **2,5** (ordinul de mărime estimat pentru SARS-CoV-2 la început)

Pragul epidemic (Holban: „cel mai important factor în epidemiologie”): dacă $R_0 < 1$, fiecare bolnav infectează mai puțin de o persoană și boala dispare; dacă $R_0 > 1$, se răspândește.

Pragul: când crește și când scade epidemia

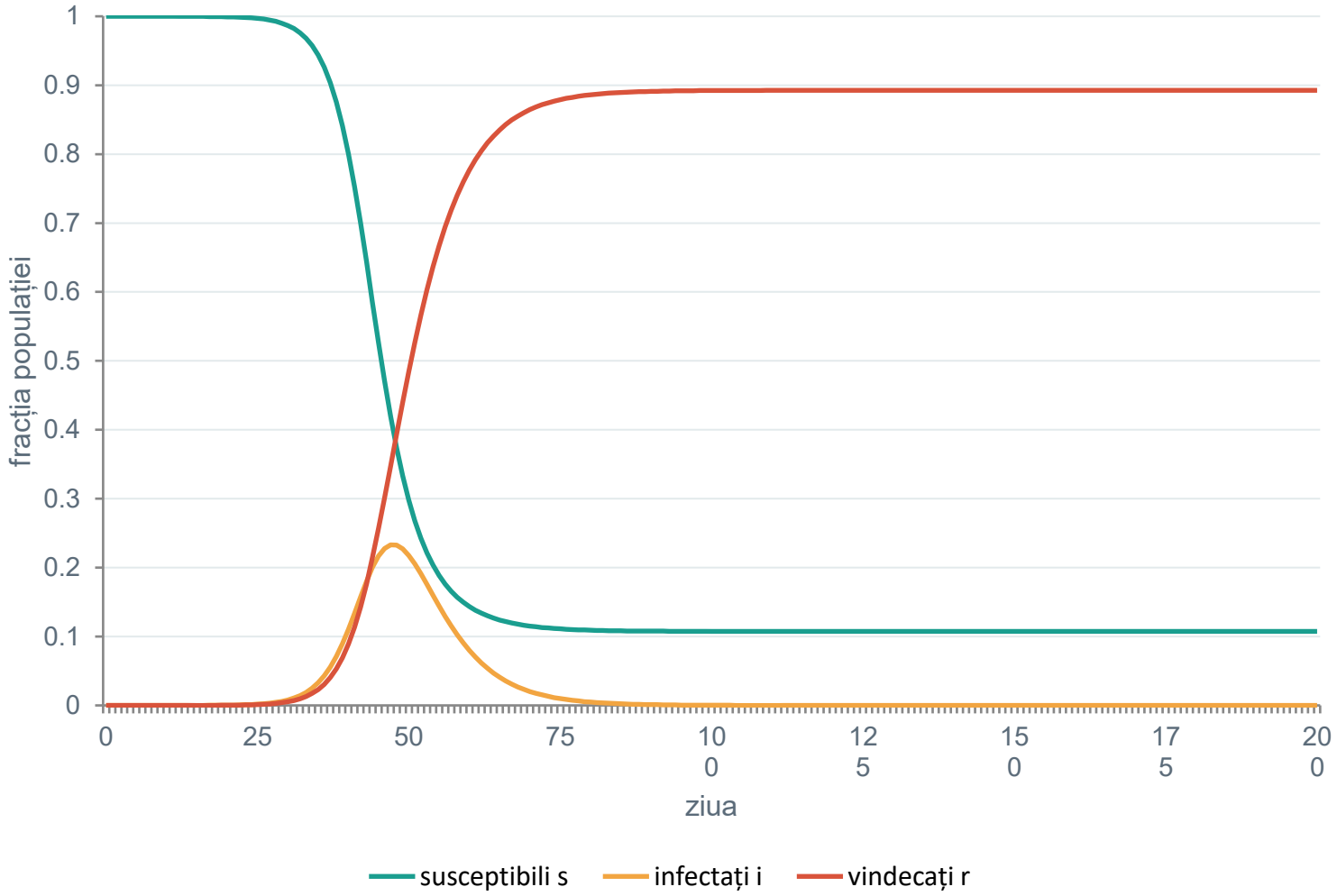
- Din ecuație: $i' = \gamma \cdot i \cdot (R_0 \cdot s - 1)$ – infectații cresc **doar dacă $R_0 \cdot s > 1$**
- **R efectiv** = $R_0 \cdot s$: scade pe măsură ce susceptibilii se epuizează (sau prin vaccinare, distanțare)
- Vârful epidemiei: exact când $s = \mathbf{1/R_0}$ (R efectiv = 1); după vârf, cazurile scad, dar epidemia continuă
- **Imunitatea de grup**: fracția imună necesară pentru ca o epidemie să nu mai poată porni: $1 - 1/R_0$
- R_0 nu este o constantă a virusului: depinde de comportament, densitate, sezon, măsuri

L

Același prag

La Lotka–Volterra, prădătorii cresc doar dacă prada depășește γ/δ. Este aceeași logică: „hrana” epidemiei sunt susceptibilii.

O epidemie SIR: R_0 = 2,5, durata bolii 5 zile, un caz la un milion



- Vârful: ziua **47**, cu **23,3%** infectați simultan
- La vârf s ≈ 0,42 ≈ $1/R_0$ = 0,4
- La final: **89,3%** au trecut prin boală
- ~11% nu se infectează niciodată

SIR în Python: zece linii

```
from scipy.integrate import solve_ivp
import numpy as np

beta, gamma = 0.5, 0.2                # R0 = 2,5

def sir(t, z):
    s, i, r = z
    return [-beta*s*i, beta*s*i - gamma*i, gamma*i]

sol = solve_ivp(sir, [0, 200], [1 - 1e-6, 1e-6, 0],
                t_eval=np.arange(201), rtol=1e-9)
s, i, r = sol.y
print(i.argmax(), i.max(), r[-1])   # 47  0,233  0,893
```

- Exact structura de la Lotka–Volterra
- Fracții: s + i + r = 1 (verificare!)
- Toleranță strictă: valorile inițiale sunt mici
- Schimbați β: vedeți pragul $R_0 = 1$

R_0, imunitatea de grup și mărimea finală

R_0	Exemplu (ordin de mărime)	Imunitate de grup $1 - 1/R_0$	Infectați la final, fără măsuri
1,5	gripa sezonieră	33%	58%
2,0	gripa pandemică 1918	50%	80%
2,5	SARS-CoV-2 (tulpina inițială)	60%	89%
3,0	variante ulterioare	67%	94%
4,0	–	75%	98%
15	rujeola	93%	≈ 100%

- Mărimea finală depășește întotdeauna pragul de imunitate: epidemia „trage” peste prag din inerție (cei infectați la vârf mai infectează)
- De aceea vaccinarea până la prag **înainte** de epidemie este mult mai eficientă decât imunitatea obținută prin boală

Ecuația mărimii finale

$$\ln s_\infty = R_0 \cdot (s_\infty - 1)$$

deducere împărțim s’ la r’: ds/dr = −R_0·s ⟹ s = e^(−R_0·r); la final r_∞ = 1 − s_∞

rezolvare nu are formulă; o rezolvăm numeric (brentq): R_0 = 2,5 ⟹ s_∞ ≈ 0,107, deci **89,3%** infectați

verificare simularea cu solve_ivp dă aceeași valoare – un test excelent pentru cod (S6)

Un rezultat remarcabil: mărimea finală depinde **doar de R_0**, nu de β și γ separat. Viteza epidemiei depinde însă de fiecare în parte.

Verificare

? O boală are $\mathbf{R_0 = 4}$. Ce fracție din populație trebuie să fie imună pentru a împiedica o epidemie?

A 25%

B 50%

C 75%

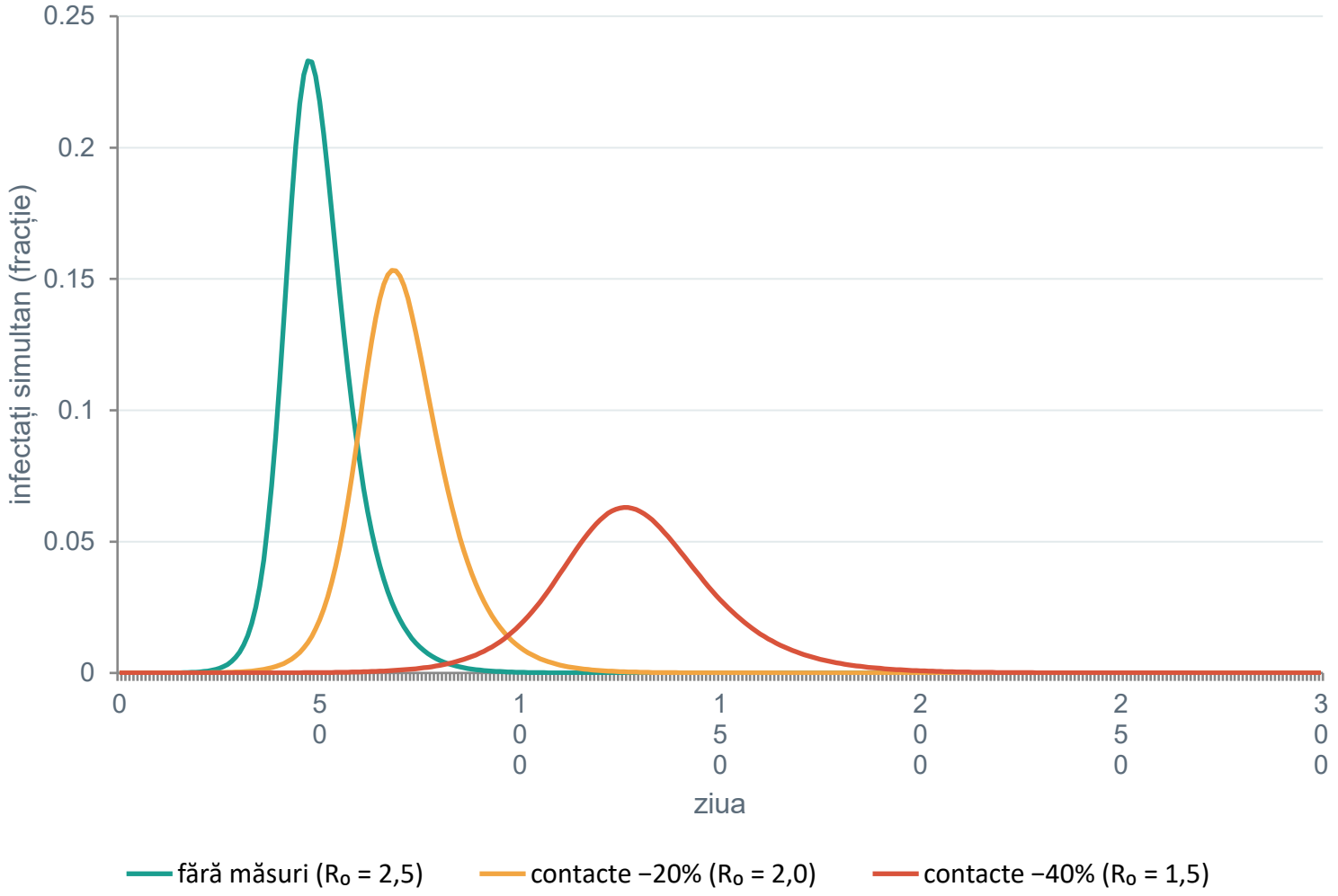
D 100%

Ce putem schimba

Intervenții și extensii

1. „Aplatizarea curbei”: reducerea lui β
2. Modelul SEIR: perioada de incubație
3. Epidemii stocastice în populații mici
4. Lecții din COVID-19

„Aplatizarea curbei”: contacte reduse de la început



- Vârful scade **și** se mută mai târziu
- −40%: vârf de 6,3% în loc de 23,3%
- Aria de sub curbă scade mai puțin decât vârful
- Câștigul principal: spitalele nu sunt depășite

Efectul reducerii contactelor

Scenariul	R_0	Ziua vârfului	Vârf (simultan)	Total infectați
fără măsuri	2,5	47	23,3%	89%
contacte −20%	2,0	68	15,3%	80%
contacte −40%	1,5	126	6,3%	58%

- Câștigul de timp contează: vaccinuri, tratamente, pregătirea spitalelor
- Dacă măsurile se ridică devreme, susceptibilii rămași permit un **al doilea val** – modelul arată de ce

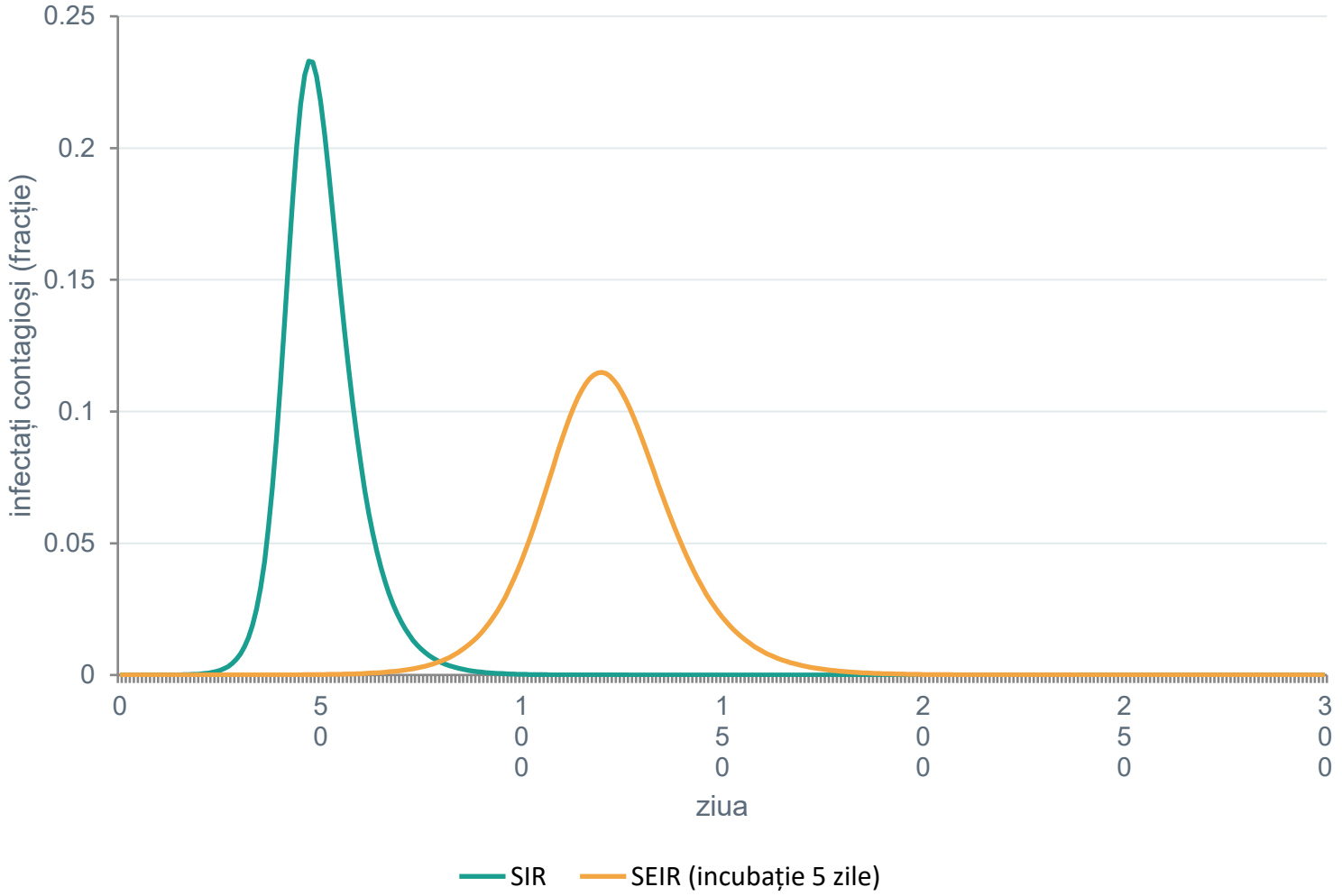
Modelul SEIR: perioada de incubație

$$s' = -\beta si, \quad e' = \beta si - \sigma e, \quad i' = \sigma e - \gamma i, \quad r' = \gamma i$$

E	**expuși**: infectați, dar încă necontagioși; trec în I cu rata σ (incubație medie 1/σ zile)
exemplul	β = 0,5, γ = 0,2, 1/σ = 5 zile: același R_0 = 2,5 ca înainte
rata de creștere	inițial: SIR 0,30/zi (dublare la 2,3 zile), SEIR 0,116/zi (dublare la 6,0 zile)

Consecință pentru calibrare: aceeași rată de creștere observată corespunde unor R_0 diferite în SIR și SEIR. **Structura** modelului contează, nu doar parametrii.

SIR vs SEIR cu același $R_0 = 2,5$



- Vârful SEIR: ziua **120** (SIR: 47)
- Vârf mai mic: 11,5% vs 23,3%
- Același total final: **89,3%**
- Mărimea finală depinde doar de R_0

Extensii ale modelului de bază

1 SIRS

Imunitatea scade în timp (R → S): epidemii recurente, ca gripa sezonieră.

2 Natalitate și mortalitate

Noi susceptibili intră continuu: boala devine **endemică**, cu oscilații amortizate.

3 Vaccinare

Un flux S → R; strategii: înainte de sezon, în inel, pe grupe de risc.

4 Structură pe vârstă

Matrice de contacte între grupe (școală, muncă, acasă): cine pe cine infectează.

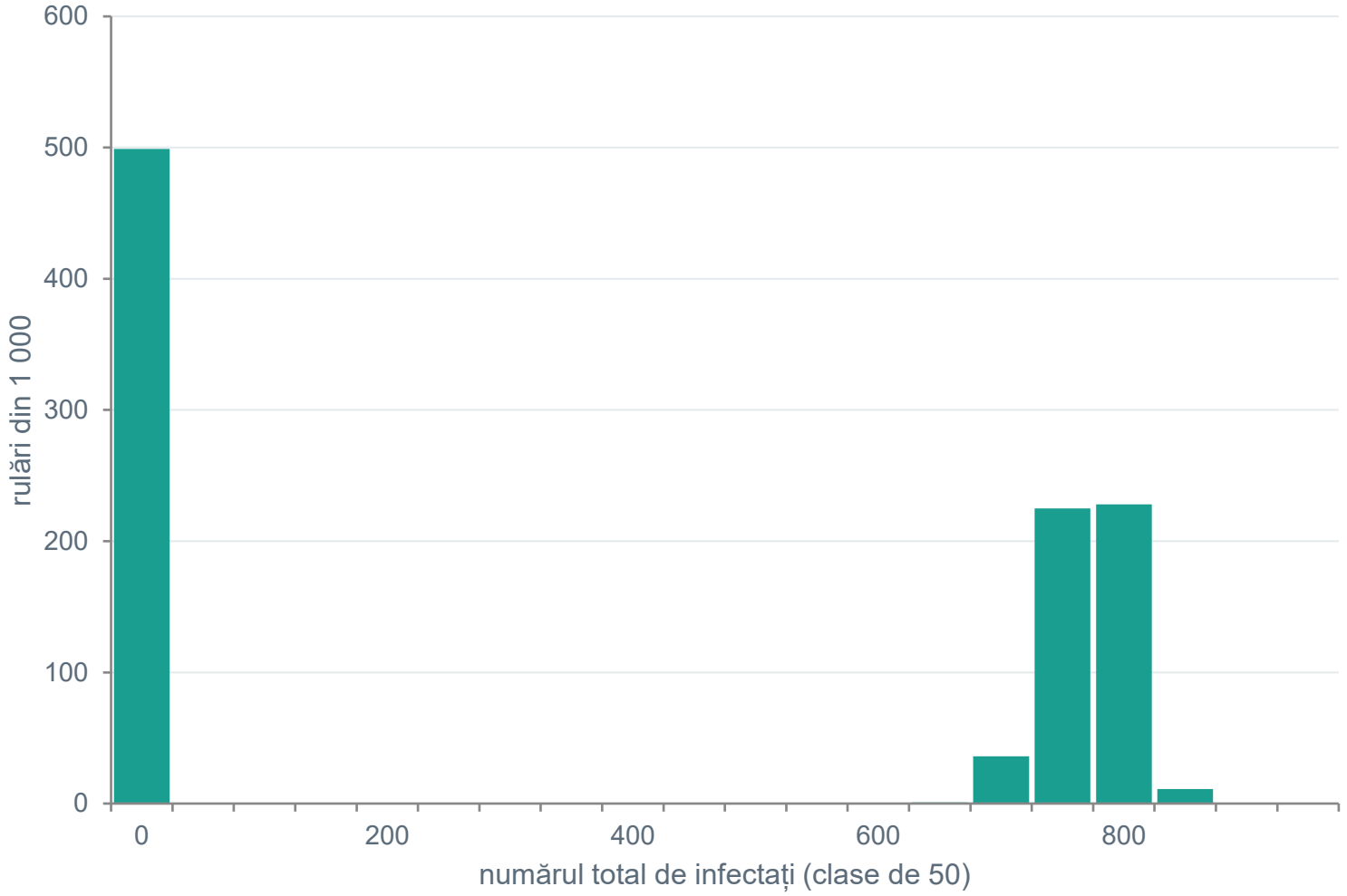
5 Spațiu și rețele

Orașe legate prin călătorii; rețele de contacte; modele cu agenți (S9).

6 Aleator

Populații mici: o epidemie poate dispărea din întâmplare – slide-ul următor.

SIR stocastic (Gillespie): N = 1 000, R_0 = 2, un singur caz inițial



- **50,1%** din rulări: focar mare
- Restul: boala dispare după câteva cazuri
- Teoria: $1 - 1/R_0 = 50\%$
- Focarele mari: în medie 79,6% (deterministic: 79,7%)

Lecțiile modelului stocastic

- Distribuția este **bimodală**: fie câteva cazuri, fie ~80% din populație – media (40%) nu descrie niciun scenariu real
- Probabilitatea de stingere timpurie ≈ $1/R_0$: primul bolnav se poate vindeca înainte să infecteze pe cineva (proces de ramificare, S3)
- Când focarul pornește, mărimea lui seamănă cu predicția deterministă (79,6% vs 79,7%)
- Algoritmul **Gillespie** (1976): timpul până la următorul eveniment ~ Exp(suma ratelor) – DES din S5 aplicat la o EDO
- Pentru populații mici (o școală, un spital, un sat) sau pentru începutul unei epidemii, aleatorul **nu** poate fi ignorat

? Ce model alegem?

EDO pentru populații mari și decizii agregate; stocastic pentru populații mici, focare incipiente și probabilități de eradicare.

Lecții din COVID-19: modele în decizii publice

- **16 martie 2020**, Imperial College, Raportul 9: fără măsuri, ~**510 000** de decese în Marea Britanie și ~**2,2 milioane** în SUA – raportul a contribuit la deciziile de carantină
- Erau **scenarii** condiționate („dacă nu facem nimic”), nu prognoze; au fost adesea citite greșit ca predicții
- Incertitudinea parametrilor era mare: R_0 estimat inițial la ~2,2 (interval 1,4–3,9; Li et al., 2020), letalitatea incertă
- Codul CovidSim a fost publicat și criticat; o verificare independentă (CODECHECK, 2020) a **reprodus** rezultatele raportului
- Comportamentul se schimbă ca răspuns la epidemie (și la prognoze): modelul influențează sistemul pe care îl modelează

C **Lecția principală**

Un model în decizii publice are nevoie de: scenarii explicite, intervale de incertitudine, cod reproductibil și comunicare clară a presupunerilor – exact temele din S6.

Modelele în pandemie: ce a mers și ce nu

Ce au făcut bine

- au arătat **ordinul de mărime** al riscului și urgența
- au comparat intervenții: școli, distanțare, carantină
- au estimat capacitatea necesară în spitale
- au explicat pragul $R < 1$ și efectul întârzierii măsurilor

Unde au avut probleme

- parametri foarte incerți la început (identificabilitate!)
- comportamentul oamenilor s-a schimbat mai repede decât modelele
- date zgomotoase: testare variabilă, raportare întârziată
- scenarii comunicate ca prognoze, fără intervale

Concluzia multor analize ulterioare: modelele simple au fost utile pentru **înțelegere** și comparații între scenarii; prognozele precise pe termen lung au fost mult mai puțin fiabile – exact ce ne așteptăm de la un sistem care se adaptează.

Verificare

? Măsurile de distanțare au redus R efectiv sub 1, iar cazurile scad. Ce prezice modelul SIR dacă măsurile se ridică brusc, când doar **20%** din populație este imună ($R_0 = 2,5$)?

A epidemia nu revine, pentru că cazurile au scăzut

B apare un nou val, pentru că $R_0 \cdot s = 2,5 \cdot 0,8 = 2 > 1$

C cazurile scad mai repede

D nu se poate spune nimic fără date noi

De la model la date

Calibrarea parametrilor

1. Problema: alegem θ astfel încât modelul să reproducă datele
2. Gripa dintr-un internat englez (1978): SIR calibrat
3. Identificabilitate: ce pot și ce nu pot spune datele timpurii
4. Lotka–Volterra pe datele Hudson’s Bay

Problema calibrării

- Avem date d_k la momentele t_k și un model care dă $m_k(\theta)$ pentru parametrii θ (de ex. β și γ)
- Căutăm θ care minimizează eroarea, de obicei **suma pătratelor**: $SSE(\theta) = \Sigma\ (m_k(\theta) - d_k)^2$
- Fiecare evaluare a lui SSE înseamnă **o simulare completă** (solve_ivp) – calibrarea este o buclă de optimizare în jurul simulatorului
- Unelte: scipy.optimize.least_squares, minimize; pentru incertitudine: bootstrap, profile, metode bayesiene (MCMC din S3)
- Rezultatul are nevoie de verificări: reziduuri, intervale pentru parametri, **validare pe date nefolosite** (S6)

M **Legătura cu ML**

Calibrarea este „antrenarea” unui model mecanistic: puțini parametri cu sens fizic în loc de milioane fără sens. Aceleași capcane: supraajustare, date insuficiente.

Datele: gripă într-un internat din nordul Angliei (BMJ, 1978)

Ziua	1	2	3	4	5	6	7	8	9	10	11	12	13	14
În pat	3	8	26	76	225	298	258	233	189	128	68	29	14	4

- **763** de elevi într-o populație închisă; epidemia a pornit de la un elev întors din vacanță; raportul menționează **512** elevi care au stat la pat
- Un set de date clasic pentru SIR: populație mică, închisă, fără intervenții, numărătoare zilnică
- Observăm „infectații” prin elevii **în pat** – o aproximare: modelul și măsurătoarea nu sunt exact aceeași mărime

Alegerea funcției obiectiv

1 Suma pătratelor

$\Sigma\,(m - d)^2$: simplă, implicită; dar zilele cu multe cazuri **domină** potrivirea.

2 Logaritmi

$\Sigma\,(\ln m - \ln d)^2$: erori **relative**; potrivită când datele variază pe ordine de mărime (iepuri și râși).

3 Verosimilitate

Număr de cazuri ~ Poisson sau binomial negativ: modelează explicit zgomotul observațiilor (S6, MLE).

4 Ponderi

Mai multă încredere în unele date (de ex. spitalizări) decât în altele (cazuri testate).

5 Mai multe serii

Cazuri, spitalizări, decese – calibrate simultan, cu ponderi alese atent.

6 Regularizare

Penalizăm valorile improbabile ale parametrilor: un pas spre metodele bayesiene.

Calibrarea în Python: solve_ivp + least_squares

```
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

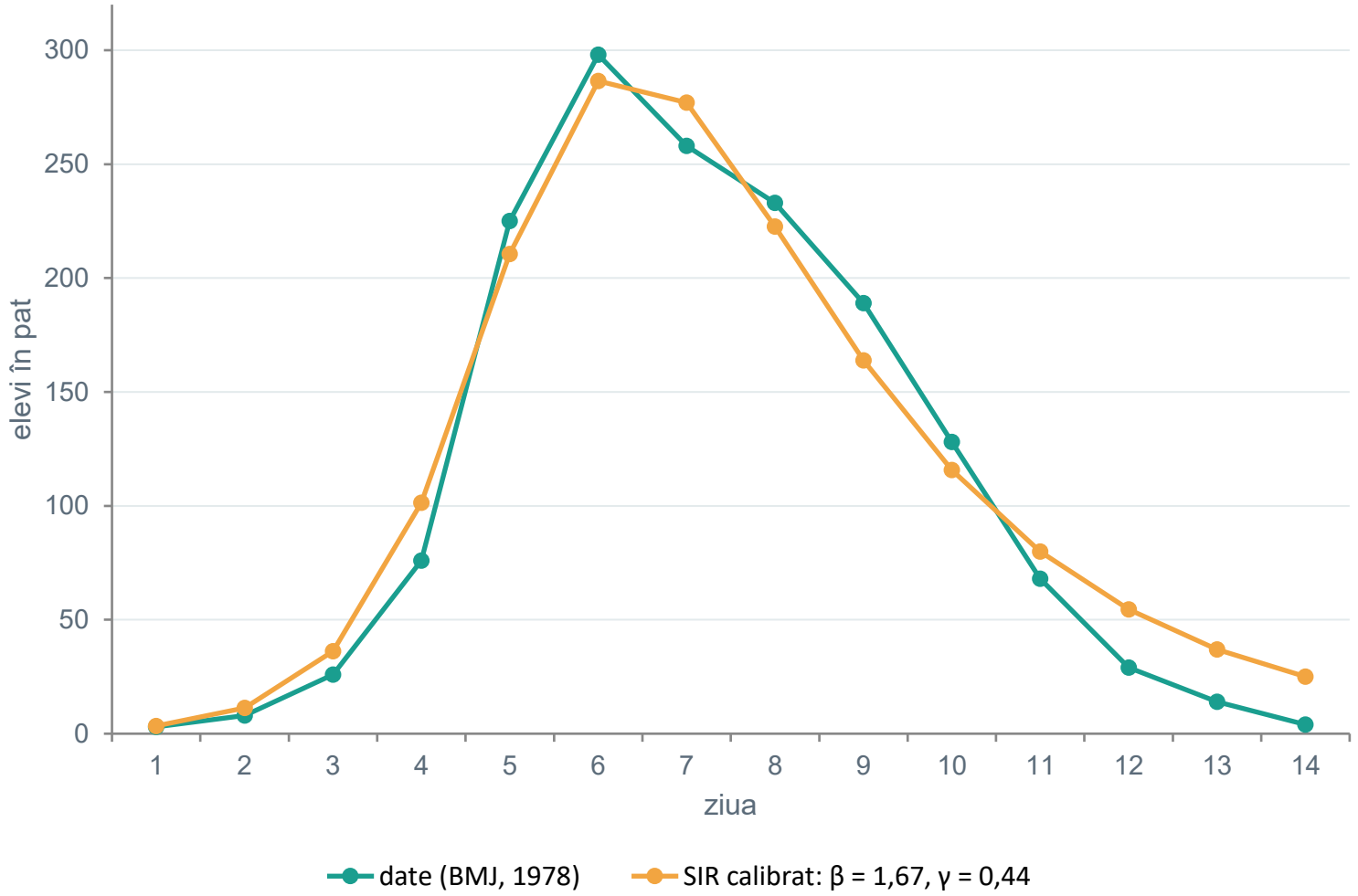
N = 763
d = np.array([3, 8, 26, 76, 225, 298, 258,
              233, 189, 128, 68, 29, 14, 4])
t = np.arange(1, 15)

def model(p):
    b, g = p
    f = lambda _, z: [-b*z[0]*z[1]/N,
                      b*z[0]*z[1]/N - g*z[1], g*z[1]]
    return solve_ivp(f, [0, 14], [N-1, 1, 0],
                     t_eval=t, rtol=1e-8).y[1]

fit = least_squares(lambda p: model(p) - d, [1, 0.5])
b, g = fit.x   # 1,669  0,443
```

- model(p): o simulare completă
- least_squares minimizează Σ reziduu²
- start: β = 1, γ = 0,5
- câteva zeci de simulări până la optim

Modelul calibrat vs datele



- R_0 = **3,76**
- Durata bolii: $1/\gamma \approx$ **2,3 zile**
- Eroarea medie (RMSE): 17 elevi
- Vârf prezis: 292 (observat: 298)

Rezultatele calibrării și incertitudinea lor

Mărimea	Valoarea	Comentariu
β (rata de infectare)	1,669 / zi	contacte × probabilitatea de transmitere
γ (rata de vindecare)	0,443 / zi	$1/\gamma \approx 2{,}3$ zile la pat
$R_0 = \beta/\gamma$	**3,76**	interval bootstrap 95%: [3,48; 3,91]
Infectați la final (ecuația mărimii finale)	744 din 763	raportul BMJ: 512 elevi la pat
RMSE	17,2 elevi	cozile prezise sunt prea groase (ziua 14: 25 vs 4)

- Intervalul pentru R_0 e îngust pentru că avem **toată** epidemia, inclusiv vârful și declinul
- Diferența 744 vs 512 este o problemă de **validare**: nu toți infectații ajung la pat, sau modelul SIR simplifică prea mult (lipsesc incubația și perioada de convalescență)

Incertitudinea parametrilor: bootstrap pe reziduuri

```
rng = np.random.default_rng(11)
fit_curve = model(fit.x)
res = d - fit_curve                      # reziduurile
R0s = []
for _ in range(200):
    d_b = fit_curve + rng.choice(res, len(res))   # date „noi”
    f_b = least_squares(lambda p: model(p) - d_b, fit.x)
    R0s.append(f_b.x[0] / f_b.x[1])

print(np.percentile(R0s, [2.5, 97.5]))   # [3,48  3,91]
```

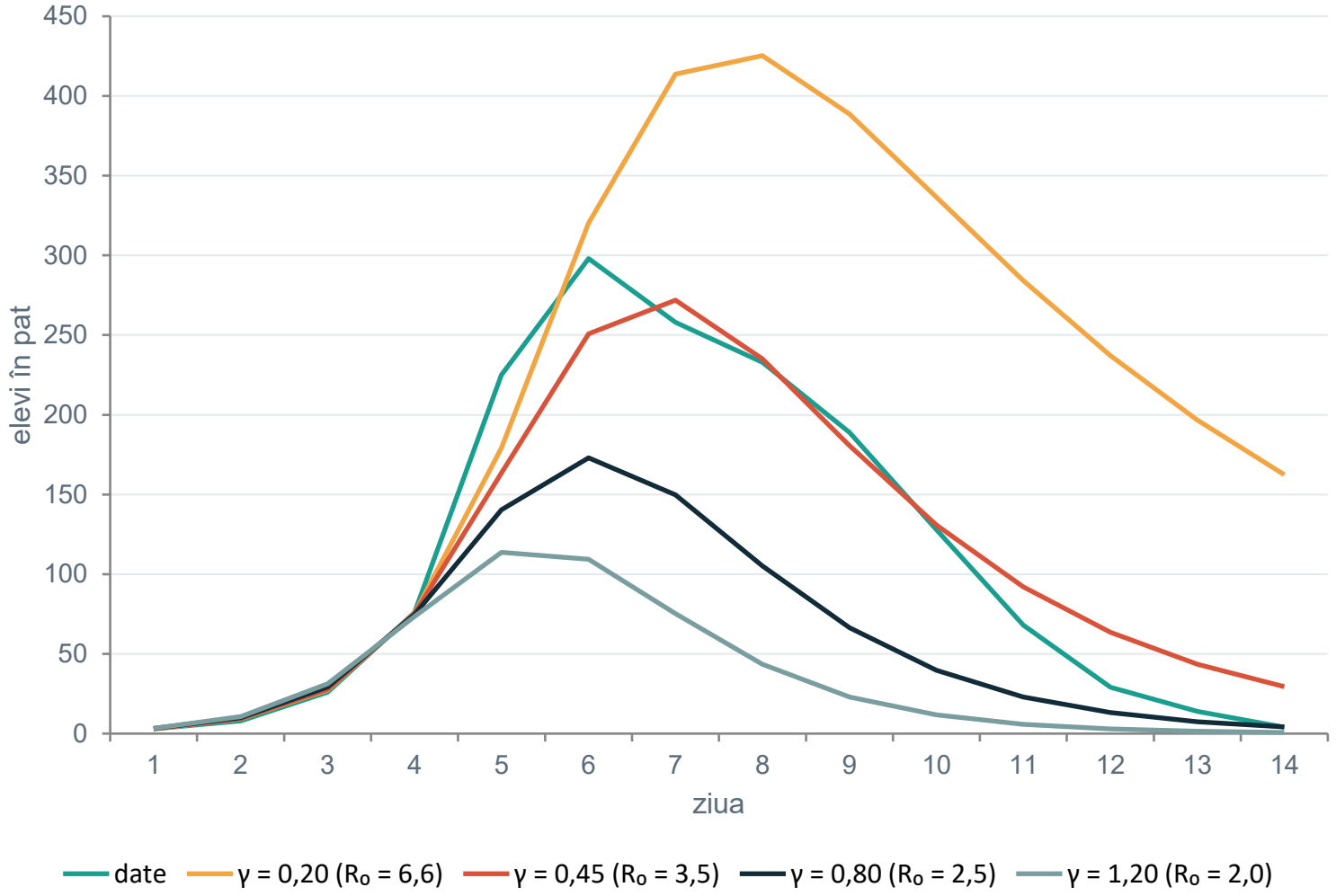
- Generăm seturi de date plauzibile
- Recalibrăm pe fiecare (200 de calibrări)
- Distribuția lui R_o dă intervalul
- Aceeași idee ca bootstrap-ul din S6

Identificabilitate: calibrare doar pe primele 4 zile

γ fixat	β calibrat	R_0	SSE pe zilele 1–4	Vârf prezis
0,20	1,318	6,59	3,2	425
0,45	1,583	3,52	6,4	272
0,80	1,961	2,45	15,7	173
1,20	2,403	2,00	40,1	114

- Toate combinațiile reproduc la fel de bine primele 4 zile: $\beta - \gamma \approx 1{,}12 \ldots 1{,}20$ (rata de creștere) este practic aceeași
- Dar R_0 variază de la 2 la 6,6, iar vârful prezis de la 114 la 425 de elevi: datele timpurii **nu pot** separa β de γ

Patru modele care se potrivesc la fel de bine pe primele 4 zile



- Primele 4 zile: curbele **coincid**
- După ziua 5: diverg complet
- Doar γ ≈ 0,45 se potrivește apoi cu datele
- Informația despre γ trebuie să vină din altă parte

Lecția identificabilității

- Un parametru este **identificabil** dacă datele disponibile îl determină; în faza exponențială, doar **β − γ** este identificabil
- Calibrat liber pe zilele 1–5, optimizatorul ajunge la $\gamma = 0{,}01$ (limita impusă) și $R_0 \approx 116$: un vârf prezis de **725** de elevi, față de 298 real
- Soluția: informație **independentă** pentru γ (durata bolii din studii clinice) sau R_0 din alte epidemii
- La începutul pandemiei din 2020, aceeași problemă: creșterea era cunoscută, dar durata infecțiozității și proporția asimptomaticilor nu
- Regula: raportați intervale pentru parametri, verificați corelațiile dintre ei și **validați** pe date ulterioare

B

Același lucru la Bass

Primele luni de adoptare a unei aplicații determină slab mărimea pieței M și vârful – exact ca R_0 la o epidemie.

Verificare

? Avem doar primele zile ale unei epidemii, în faza de creștere exponențială. Ce parametru pot estima **sigur** datele?

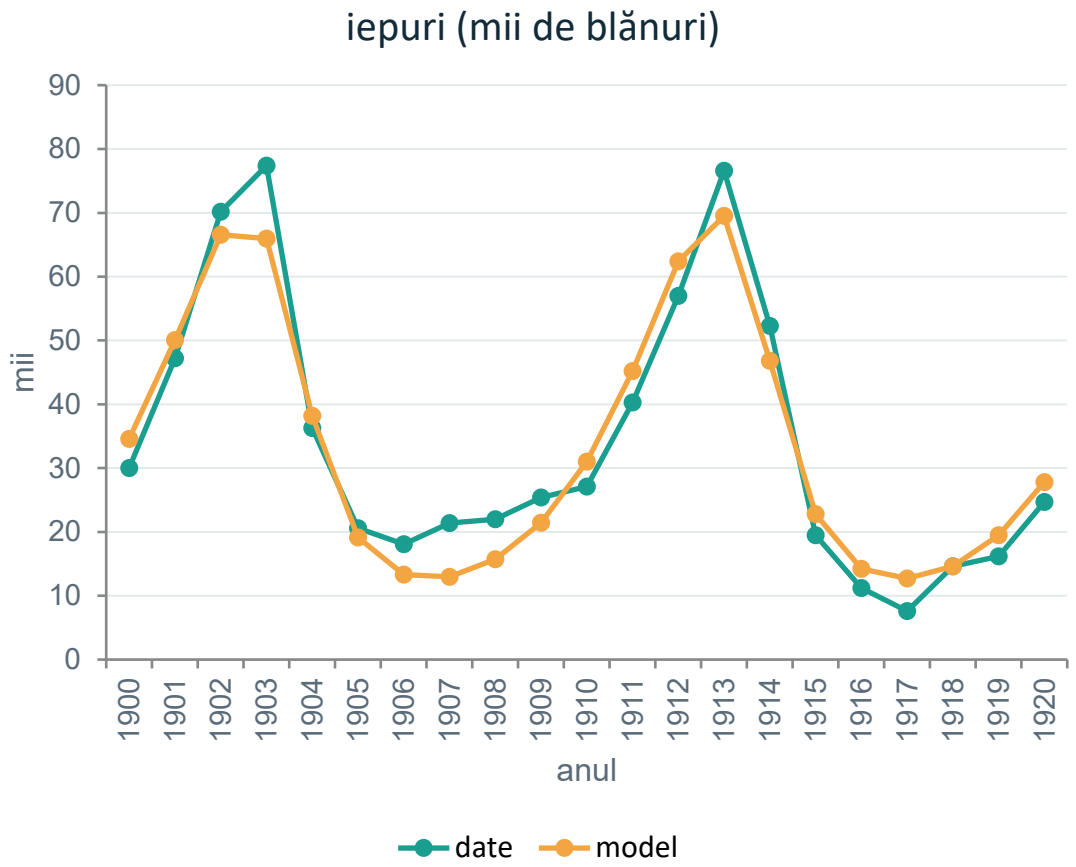
A R_0

B durata bolii $1/\gamma$

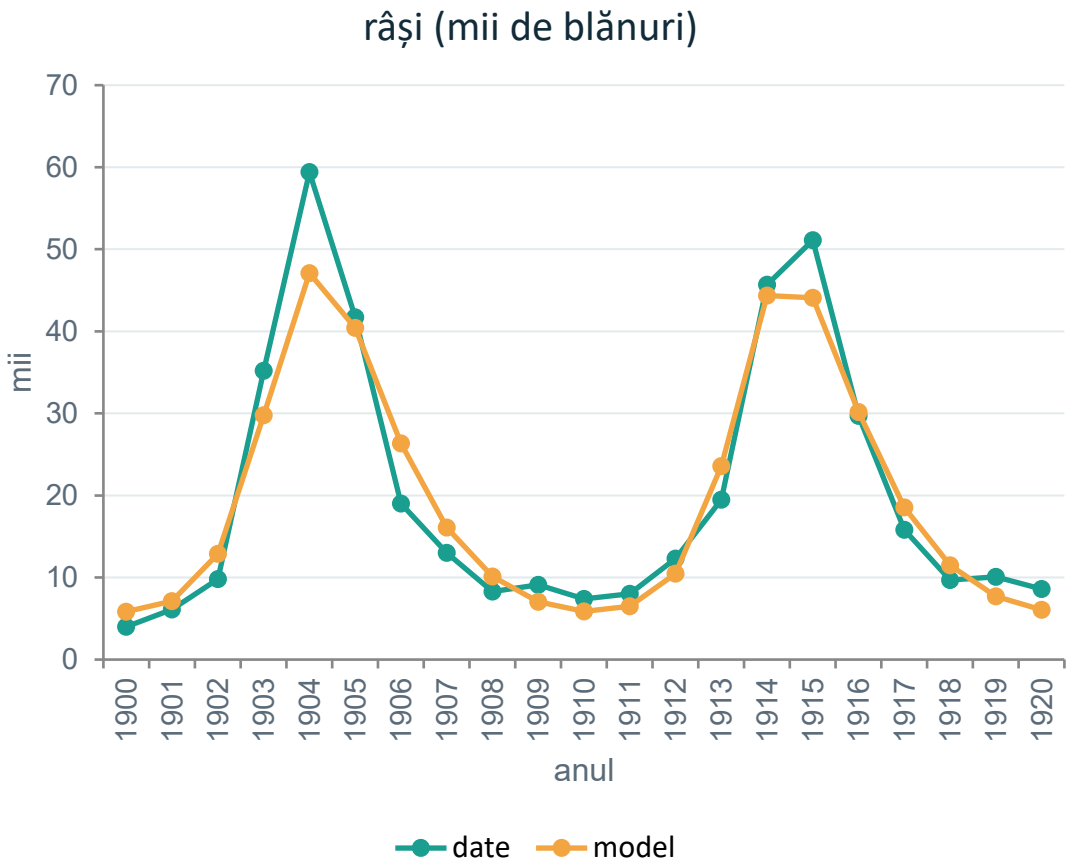
C rata de creștere $\beta - \gamma$

D mărimea finală a epidemiei

Lotka–Volterra calibrat pe datele Hudson’s Bay Company (1900–1920)



prada: vârfuri în 1903 și 1913



prădătorii: vârfurile vin cu ~1 an mai târziu

Ce aflăm din calibrarea pradă–prădător

- Parametri estimați: $\alpha = 0{,}54$, $\beta = 0{,}027$, $\gamma = 0{,}80$, $\delta = 0{,}024$ (pe an)
- Echilibrul $(\gamma/\delta, \alpha/\beta) \approx (34; 20)$ mii; perioada liniară $2\pi/\sqrt{(\alpha\gamma)} \approx$ **9,6 ani** – ciclul observat este de ~10 ani
- Am potrivit **logaritmii**: populațiile variază de 10 ori, iar erorile relative contează mai mult decât cele absolute
- Limitări: blănurile cumpărate nu sunt populații (depind de prețuri, de vânători); alte specii și clima lipsesc din model
- Puntea spre S10: când nu știm ecuațiile, le putem **învăța** din date (SINDy, neural ODE, PINN)

? Ce model e bun?

Patru parametri cu sens biologic, un ciclu de ~10 ani explicat de mecanism: modelul nu e exact, dar e **util** – și, mai ales, testabil.

Rezumatul săptămânii

- ✓ Model dinamic: stare, intrări, parametri; **ecuații de bilanț**; logistică, răcire, stocuri și fluxuri, întârzieri → oscilații
- ✓ Integrare: **Euler** (ordin 1), Heun, **RK4** (ordin 4); stabilitate $h < 2/\lambda$; rigiditate → metode implicite; solve_ivp adaptiv
- ✓ Lotka–Volterra: același pas, Euler explodează, RK4 păstrează orbitele; verificăm cu mărimi conservate
- ✓ **SIR**: $R_0 = \beta/\gamma$, vârf la $s = 1/R_0$, imunitate de grup $1 - 1/R_0$; SEIR, stocastic, lecțiile COVID-19
- ✓ **Calibrare**: least_squares în jurul simulatorului; identificabilitate, incertitudine, validare

Săptămâna viitoare

Automate celulare, particule, simulare de înaltă performanță

- Automate celulare: reguli locale, comportament global (Game of Life, trafic)
- Sisteme de particule: integrarea Verlet, grile, Barnes–Hut
- Performanță: vectorizare, paralelism, GPU

Am integrat sisteme cu 2–4 variabile. Ce facem când avem un milion de particule sau o grilă cu un miliard de celule?

Bibliografie pentru săptămâna 7

- Ș. Holban – Tehnici de modelare și simulare, cap. 6: modele matematice, Lotka–Volterra, metode numerice; cap. 7: modelul Kermack–McKendrick (materiale de curs)
- Prof. Cernazean – Modelling and Simulation, Lecture 3: Dynamical systems and numerical integration (materiale de curs)
- E. Hairer, S. P. Nørsett, G. Wanner – Solving Ordinary Differential Equations I, Springer, 1993; Hairer & Wanner – vol. II (stiff), 1996
- S. H. Strogatz – Nonlinear Dynamics and Chaos, ed. 2, Westview, 2015
- J. D. Sterman – Business Dynamics: Systems Thinking and Modeling for a Complex World, McGraw-Hill, 2000
- W. O. Kermack, A. G. McKendrick – A contribution to the mathematical theory of epidemics, Proc. Royal Society A, 115, 1927
- M. J. Keeling, P. Rohani – Modeling Infectious Diseases in Humans and Animals, Princeton, 2008
- N. M. Ferguson et al. – Report 9: Impact of non-pharmaceutical interventions (NPIs) to reduce COVID-19 mortality and healthcare demand, Imperial College, 2020
- F. M. Bass – A new product growth model for consumer durables, Management Science, 15(5), 1969
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5